

NoteHub 実装仕様書

システム概要

基本情報

項目	内容
システム名	NoteHub
システム種別	リアルタイム共同編集オンラインエディタ
利用対象	チームで共同作業を行うユーザー
主な機能	ユーザー認証、ワークスペース管理、メンバー管理、ドキュメント管理、共同編集、編集履歴、アカウント管理
通信方式	REST API、WebSocket
データ	PostgreSQL
一時データ	Redis

システム目的

NoteHubは、複数ユーザーが同じドキュメントをリアルタイムで共同編集できる環境を提供するワークスペース単位でメンバーとドキュメントを管理し、WebSocketを利用し編集内容を同期するドキュメント内容はRedisへ一時保存し、自動保存処理によってPostgreSQLへ永続化するDocumentVersionを利用して編集履歴を保存し、過去の状態へ復元できるようにする

悪意あるユーザーについては、まずアカウント停止を行い、調査によって悪意が確認された場合に論理削除する

停止または論理削除されたユーザーが他のワークスペースでホストを務めている場合、そのワークスペースも利用不可にする

解決する課題

課題	解決方法
編集内容が他の利用者へ反映されない	WebSocketでリアルタイム同期する
編集内容を保存し忘れる	自動保存する
過去の状態へ戻れない	DocumentVersionへ編集履歴を保存する
誰が編集しているか分からない	接続中ユーザーを編集者一覧として表示する
悪意あるユーザーをすぐに排除できない	ホストがアカウント停止を実行できるようにする
誤ってアカウントを削除する可能性がある	停止と論理削除を段階的に行う
停止済みホストのワークスペースが利用され続ける	ホストのアカウント状態からワークスペース利用可否を判定する

技術スタック

分類	技術
フロントエンド	React
フロントエンド言語	TypeScript
ビルドツール	Vite
CSS	Tailwind CSS
エディタ	Monaco Editor
バックエンド	Go
Webフレームワーク	Echo
ORM	GORM
データベース	PostgreSQL
キャッシュ	Redis
認証	JWT
パスワード	bcrypt

分類	技術
リアルタイム通信	Gorilla WebSocket
API	REST API、WebSocket
インフラ	Docker、Docker Compose
バックエンドテスト	Go testing
フロントエンドテスト	Vitest
CI/CD	GitHub Actions
Access Token	JWT
JWT署名	HS256
Refresh Token生成	crypto/rand
Refresh Token Hash	SHA-256
Password Hash	bcrypt
CSRF対策	Double Submit Cookie
Token失効管理	PostgreSQL、Redis

アプリケーション構成

バックエンド構成

バックエンドは以下の処理順を基本とする

```
1.Router
2.Middleware
3.Controller
4.UseCase
5.Repository
6.PostgreSQLまたはRedis
```

WebSocketは以下の処理順を基本とする

```
1.WebSocket Router
2.WebSocket認証
3.WebSocket Controller
4.UseCase
5.Repository
6.WebSocket Hub
```

各層の責務

層	責務
Router	URLとControllerを関連付ける
Middleware	認証、CSRF、CORS、Rate Limit、ログ処理を行う
Controller	リクエストを受け取り、DTOへ変換してUseCaseを呼び出す
UseCase	業務処理、権限確認、トランザクション、処理順序を管理する
Repository	PostgreSQLへのデータアクセスを行う
Redis	Token管理、Rate Limit、一時編集内容、自動保存待ちデータを管理する
WebSocket	接続管理、イベント受信、イベント配信、強制切断を行う
batch	自動保存、バックアップなどの定期処理を行う
Model	DBモデルと定数を定義する
DTO	APIの入力値と出力値を定義する
Config	環境変数とアプリケーション設定を管理する
Crypto	JWT、bcrypt、Token生成、ハッシュ処理を管理する

Crypto詳細責務

項目	内容
Password Hash	bcryptを使用してPasswordをHash化する
Password照合	入力Passwordと保存済みHashを照合する
Access Token生成	JWT形式のAccess Tokenを生成する
Access Token検証	JWTの署名、Claims、有効期限を検証する
Refresh Token生成	暗号学的に安全なランダム値を生成する
Refresh Token Hash	SHA-256でHash化する
CSRF Token生成	暗号学的に安全なランダム値を生成する

レイヤー別実装方針

Router

責務

項目	内容
HTTP Router	REST APIのURLとControllerを関連付ける
WebSocket Router	WebSocket接続URLとControllerを関連付ける
Middleware適用	認証、CSRF、Rate Limitなどをルート単位で適用する
APIグループ	公開API、認証必須API、ホスト専用APIを分ける

禁止事項

禁止事項	内容
業務ロジック	Router内で権限判定やDB更新を行わない
DBアクセス	Repositoryを直接呼び出さない
Redisアクセス	Redisへ直接アクセスしない
レスポンス生成	Controller以外で通常のAPIレスポンスを生成しない

Middleware

責務

Middleware	処理内容
AuthMiddleware	Access Tokenを検証する
CSRFMiddleware	CSRF CookieとX-CSRF-Tokenを照合する
CORSMiddleware	許可されたフロントエンドのみ通信を許可する
RateLimitMiddleware	Token Bucket方式でリクエスト回数を制限する
RequestIdMiddleware	リクエストごとにRequest IDを付与する
LoggingMiddleware	Method、Path、Status、処理時間などを記録する
RecoveryMiddleware	Panicを捕捉し、500エラーへ変換する

AuthMiddleware処理

Access Tokenを検証し、認証済みUserとToken情報をContextへ保存する

- 1.Authorizationヘッダー取得
- 2.Bearer Token取得
- 3.JWT署名検証
- 4.有効期限確認
- 5.user_id取得
- 6.User取得
- 7.User.status確認

8. Contextへ認証ユーザーを保存
9. 次の処理へ進む

Access Token仕様

項目	内容
形式	JWT
署名アルゴリズム	HS256
有効期限	15分
保存場所	ブラウザメモリ
LocalStorage	保存禁止
SessionStorage	保存禁止
Cookie	保存しない
DB保存	行わない
送信方法	Authorization Header
Header形式	Authorization: Bearer {AccessToken}

JWT Claims

Claim	内容
iss	Token発行者
aud	Token利用対象
sub	User ID
jti	Access Token固有ID
iat	発行日時
exp	有効期限
ver	User.auth_version

処理フロー

1. Authorization Headerを取得
2. Authorization HeaderがBearer形式であることを確認
3. Bearerの後ろからAccess Tokenを取得
4. JWT形式が正しいことを確認
5. 署名アルゴリズムがHS256であることを確認
6. JWT_SECRETを使用して署名を検証
7. issがJWT_ISSUERと一致することを確認
8. audがJWT_AUDIENCEと一致することを確認
9. expが現在日時より後であることを確認
10. subからUser IDを取得
11. jtiからAccess Token IDを取得
12. verから認証Versionを取得
13. Redisでjtiが失効済みでないことを確認
14. UserRepositoryからUserを取得
15. User.statusがactiveであることを確認
16. JWTのverとUser.auth_versionが一致することを確認
17. Contextへ認証Userを保存
18. ContextへUser IDを保存
19. Contextへjtiを保存
20. Contextへexpを保存
21. 次の処理へ進む

Context保存値

Key	内容
auth_user	認証済みUser
user_id	認証User ID

Key	内容
access_token_jti	Access Token固有ID
access_token_exp	Access Token有効期限
auth_version	JWTの認証Version

拒否条件

条件	ErrorCode	HTTP Status
Authorization Headerなし	ACCESS_TOKEN_REQUIRED	401
Bearer形式不正	ACCESS_TOKEN_INVALID	401
JWT形式不正	ACCESS_TOKEN_INVALID	401
HS256以外	ACCESS_TOKEN_INVALID	401
署名不正	ACCESS_TOKEN_INVALID	401
iss不一致	ACCESS_TOKEN_INVALID	401
aud不一致	ACCESS_TOKEN_INVALID	401
有効期限切れ	ACCESS_TOKEN_EXPIRED	401
sub不正	ACCESS_TOKEN_INVALID	401
jti不正	ACCESS_TOKEN_INVALID	401
jti失効済み	ACCESS_TOKEN_REVOKED	401
Userなし	ACCESS_TOKEN_INVALID	401
User停止中	ACCOUNT_UNAVAILABLE	401
User論理削除済み	ACCOUNT_UNAVAILABLE	401
auth_version不一致	ACCESS_TOKEN_REVOKED	401

Tokenへ含めない情報

情報	理由
Email	個人情報の不要な開示を防止
Password Hash	認証情報のため
User.status	発行後に状態が変更されるため
Workspace Role	権限変更後に古い情報が残るため
Workspace一覧	Token内の情報過多を防ぐ
Document権限	DBの所属情報を正とするため

アカウント状態判定

status	処理
active	認証成功
suspended	認証拒否
deleted	認証拒否

CSRFMiddleware

Refresh TokenをCookieで送信するAPIに対してCSRF攻撃を防止する

対象API

Method	Path	CSRF検証
POST	/api/auth/refresh	必須
POST	/api/auth/logout	必須
Authorization Headerのみを使用するAPI	各種	不要

CSRF Token仕様

項目	内容
生成方法	crypto/rand
乱数長	32バイト
文字列形式	Base64 URL Safe
Padding	なし
Cookie名	notehub_csrf_token
Header名	X-CSRF-Token
HttpOnly	false
Secure	本番環境ではtrue
比較方法	一定時間比較

処理フロー

1. CSRF Cookieを取得
2. X-CSRF-Token Headerを取得
3. CSRF Cookieが存在することを確認
4. X-CSRF-Token Headerが存在することを確認
5. CookieとHeaderの値を一定時間比較で照合
6. 一致した場合はContextへ検証済み情報を保存
7. 一致した場合は次の処理へ進む
8. 不一致の場合は403 Forbiddenを返す

Context保存値

Key	値
csrf_validated	true

拒否条件

条件	ErrorCode	HTTP Status
CSRF Cookieなし	CSRF_TOKEN_REQUIRED	403
CSRF Headerなし	CSRF_TOKEN_REQUIRED	403
Token不一致	CSRF_TOKEN_INVALID	403

Controller

責務

項目	内容
Path Parameter	workspaceId、documentId、userIdなどを取得する
Query Parameter	検索条件、ページ番号などを取得する
Request Body	JSONをRequest DTOへ変換する
入力値検証	Validatorを呼び出す
UseCase実行	適切なUseCaseを呼び出す
Response変換	UseCase結果をResponse DTOへ変換する
エラー変換	UseCase ErrorをHTTP Statusへ変換する

禁止事項

禁止事項	内容
DBアクセス	Repositoryを直接呼び出さない

禁止事項	内容
Redisアクセス	Redisを直接操作しない
業務ロジック	権限判定や状態遷移を実装しない
Transaction	Transactionを開始しない
Password処理	bcryptを直接呼び出さない
JWT発行	JWT生成処理を直接実行しない

UseCase

責務

項目	内容
業務処理	各機能の処理順序を管理する
認可	ワークスペース所属とロールを確認する
状態確認	User、Workspace、Documentなどの状態を確認する
Transaction	複数DB更新をまとめて処理する
Repository	DB操作をRepositoryへ依頼する
Redis	Redis処理をRedis用Interfaceへ依頼する
WebSocket	イベント通知や切断をWebSocket Managerへ依頼する
監査ログ	重要操作をAuditLogへ保存する
エラー変換	Repository ErrorをUseCase Errorへ変換する

処理順序

1. 入力値を検証
2. 実行ユーザーを取得
3. 実行ユーザーのアカウント状態を確認
4. 対象データを取得
5. 所属を確認
6. 権限を確認
7. 業務条件を確認
8. Transactionを開始
9. DBを更新
10. AuditLogを保存
11. TransactionをCOMMIT
12. Redisを更新
13. WebSocket通知または切断を実行
14. 処理結果を返却

WebSocket通知原則

状況	処理
DB COMMIT成功	WebSocket通知を実行する
DB COMMIT失敗	WebSocket通知を実行しない
Redis更新失敗	エラーを記録し、必要に応じて復旧処理を行う
WebSocket通知失敗	DBをRollbackせずログへ記録する

Repository

責務

項目	内容
CRUD	PostgreSQLの登録、取得、更新、削除を行う
GORM	GORMを利用してQueryを実行する
Transaction	UseCaseから渡されたTransactionを利用する
DB Error	DBエラーを共通Repository Errorへ変換する

項目	内容
Lock	必要な処理で行ロックを行う

禁止事項

禁止事項	内容
業務条件判定	アカウント停止可能かどうかを判定しない
WebSocket通知	Repositoryから通知しない
HTTP処理	HTTP Statusを返さない
DTO変換	API用DTOを生成しない

ディレクトリ構成

フロントエンド

```

frontend/
├── src/
│   ├── api/
│   ├── components/
│   │   ├── admin/
│   │   ├── auth/
│   │   ├── game/
│   │   ├── layout/
│   │   ├── matching/
│   │   ├── ranking/
│   │   ├── workspace/
│   │   └── ui/
│   ├── constants/
│   ├── contexts/
│   ├── hooks/
│   ├── lib/
│   ├── pages/
│   │   ├── admin/
│   │   ├── auth/
│   │   ├── game/
│   │   ├── matching/
│   │   └── ranking/
│   ├── router/
│   ├── styles/
│   ├── test/
│   ├── types/
│   ├── utils/
│   ├── App.tsx
│   ├── App.test.tsx
│   ├── index.css
│   ├── main.tsx
│   └── vite-env.d.ts
├── public/
├── dist/
├── node_modules/
├── .env.example
├── package.json
└── vite.config.ts

```

バックエンド

```

backend/
├── config/
│   ├── config.go
│   └── env.go
├── controller/
│   ├── auth.go
│   ├── workspace.go
│   ├── member.go
│   └── account.go

```

```

├── document.go
├── version.go
├── websocket.go
├── crypto/
│   ├── jwt.go
│   ├── password.go
│   ├── random_token.go
│   └── token_hash.go
├── db/
│   ├── postgres.go
│   ├── transaction.go
│   └── migrate.go
├── docs/
│   └── openapi.yaml
├── dto/
│   ├── auth.go
│   ├── workspace.go
│   ├── member.go
│   ├── account.go
│   ├── document.go
│   ├── version.go
│   ├── websocket.go
│   ├── response.go
│   └── error.go
├── middleware/
│   ├── auth.go
│   ├── csrf.go
│   ├── cors.go
│   ├── rate_limit.go
│   ├── request_id.go
│   ├── logging.go
│   └── recovery.go
├── migrations/
│   ├── create_users.up.sql
│   ├── create_users.down.sql
│   ├── create_workspaces.up.sql
│   ├── create_workspaces.down.sql
│   ├── create_workspace_members.up.sql
│   ├── create_workspace_members.down.sql
│   ├── create_documents.up.sql
│   ├── create_documents.down.sql
│   ├── create_document_versions.up.sql
│   ├── create_document_versions.down.sql
│   ├── create_refresh_tokens.up.sql
│   ├── create_refresh_tokens.down.sql
│   ├── create_audit_logs.up.sql
│   ├── create_users.up.sql
│   └── create_audit_logs.down.sql
├── model/
│   ├── user.go
│   ├── workspace.go
│   ├── workspace_member.go
│   ├── document.go
│   ├── document_version.go
│   ├── refresh_token.go
│   ├── audit_log.go
│   └── constants.go
├── redis/
│   ├── client.go
│   ├── rate_limit.go
│   ├── access_token_revocation.go
│   ├── document_cache.go
│   ├── websocket_session.go
│   ├── csrf.go
│   └── lock.go
├── repository/
│   ├── user.go
│   ├── workspace.go
│   ├── workspace_member.go
│   ├── document.go
│   ├── document_version.go
│   ├── refresh_token.go
│   └── audit_log.go

```

```

├── router/
│   ├── router.go
│   ├── auth.go
│   ├── workspace.go
│   ├── document.go
│   └── websocket.go
├── testutil/
│   ├── database.go
│   ├── redis.go
│   ├── user.go
│   ├── workspace.go
│   └── token.go
├── usecase/
│   ├── auth.go
│   ├── auth_helpers.go
│   ├── auth_register.go
│   ├── auth_login.go
│   ├── auth_refresh.go
│   ├── auth_logout.go
│   ├── auth_me.go
│   ├── workspace.go
│   ├── workspace_create.go
│   ├── workspace_list.go
│   ├── workspace_get.go
│   ├── workspace_update.go
│   ├── workspace_delete.go
│   ├── workspace_access.go
│   ├── member.go
│   ├── member_search.go
│   ├── member_list.go
│   ├── member_add.go
│   ├── member_remove.go
│   ├── account.go
│   ├── account_suspend.go
│   ├── account_reactivate.go
│   ├── account_delete.go
│   ├── account_workspace_lock.go
│   ├── document.go
│   ├── document_create.go
│   ├── document_list.go
│   ├── document_get.go
│   ├── document_update.go
│   ├── document_delete.go
│   ├── version.go
│   ├── version_list.go
│   ├── version_get.go
│   ├── version_create.go
│   ├── version_restore.go
│   ├── errors.go
│   └── helpers.go
├── websocket/
│   ├── hub.go
│   ├── client.go
│   ├── connection.go
│   ├── event.go
│   ├── handler.go
│   ├── editor.go
│   ├── broadcast.go
│   └── disconnect.go
├── batch/
│   ├── auto_save.go
│   ├── document_flush.go
│   ├── backup.go
│   └── cleanup.go
├── Dockerfile
├── go.mod
├── go.sum
├── main.go
└── main_test.go

```

RefreshToken Migration

```
CREATETABLE refresh_tokens (  
  id UUIDPRIMARYKEY,  
  user_id UUIDNOTNULL,  
  token_hashCHAR(64)NOTNULLUNIQUE,  
  family_id UUIDNOTNULL,  
  statusVARCHAR(20)NOTNULLDEFAULT'active',  
  expires_at TIMESTAMPTZNOTNULL,  
  created_at TIMESTAMPTZNOTNULLDEFAULTCURRENT_TIMESTAMP,  
  rotated_at TIMESTAMPTZNULL,  
  revoked_at TIMESTAMPTZNULL,  
  last_used_at TIMESTAMPTZNULL,  
  replaced_by_id UUIDNULL,  
  ip_addressVARCHAR(45)NULL,  
  user_agent TEXTNULL,CONSTRAINT fk_refresh_tokens_userFOREIGNKEY (user_id)REFERENCES users(id),CONSTRAINT fk_refresh_tokens_re  
placed_byFOREIGNKEY (replaced_by_id)REFERENCES refresh_tokens(id),CONSTRAINT chk_refresh_tokens_statusCHECK (statusIN ('activ  
e','rotated','revoked','expired'))  
)
```

Index

```
CREATE INDEX idx_refresh_tokens_user_idON refresh_tokens(user_id)CREATE INDEX idx_refresh_tokens_family_idON refresh_tokens(famil  
y_id)CREATE INDEX idx_refresh_tokens_statusON refresh_tokens(status)CREATE INDEX idx_refresh_tokens_expires_atON refresh_tokens(e  
xpires_at)CREATE INDEX idx_refresh_tokens_user_statusON refresh_tokens(user_id, status)
```

Down Migration

```
DROPTABLEIFEXISTS refresh_tokens
```

Redisファイル責務

ファイル	責務
client.go	Redis接続を管理
rate_limit.go	Token Bucket方式のRate Limitを管理
access_token_revocation.go	Access Tokenのjti失効情報を管理
document_cache.go	編集中Documentを管理
websocket_session.go	WebSocket接続情報を管理
csrf.go	CSRF関連の一時情報を管理
lock.go	多重実行防止Lockを管理

Cryptoファイル責務

ファイル	責務
jwt.go	Access Token生成、署名、検証
password.go	bcrypt Hash生成、Password照合
random_token.go	Refresh Token、CSRF Token生成
token_hash.go	Refresh TokenのSHA-256 Hash生成

ディレクトリ責務

ディレクトリ	責務
config	環境変数と設定値を読み込む
controller	HTTPとWebSocketの入力と出力を管理する
crypto	JWT、bcrypt、Token、Hashを管理する
db	PostgreSQL接続とTransactionを管理する
docs	OpenAPIなどのAPI仕様を保存する

ディレクトリ	責務
dto	Request、Response、WebSocket Eventを定義する
middleware	HTTP共通処理を管理する
migrations	DB Migrationを管理する
model	DB Modelと定数を定義する
redis	Redisへのアクセスを管理する
repository	PostgreSQLへのアクセスを管理する
router	REST APIとWebSocketのRouteを定義する
testutil	テスト共通処理を管理する
usecase	業務処理を管理する
websocket	WebSocket接続と配信を管理する
batch	自動保存、バックアップ、定期処理を管理する

Config仕様

認証設定値

環境変数	必須	初期値	内容
JWT_SECRET	必須	なし	JWT署名秘密鍵
JWT_ISSUER	任意	notehub-api	JWT発行者
JWT_AUDIENCE	任意	notehub-client	JWT利用対象
ACCESS_TOKEN_TTL_MINUTES	任意	15	Access Token有効期限
REFRESH_TOKEN_TTL_MINUTES	開発環境	30	開発用Refresh Token有効期限
REFRESH_TOKEN_TTL_DAYS	本番環境	14	本番用Refresh Token有効期限
BCRYPT_COST	任意	12	bcrypt Cost
COOKIE_SECURE	任意	true	Secure属性
COOKIE_SAME_SITE	任意	Lax	SameSite属性
COOKIE_DOMAIN	任意	空文字	Cookie Domain
LOGIN_MAX_FAILURES	任意	5	Login最大失敗回数
LOGIN_LOCK_SECONDS	任意	3600	Login制限時間
ACCESS_TOKEN_COOKIE_NAME	任意	使用しない	Access TokenはCookie保存しない
REFRESH_TOKEN_COOKIE_NAME	任意	notehub_refresh_token	Refresh Token Cookie名
CSRF_TOKEN_COOKIE_NAME	任意	notehub_csrf_token	CSRF Cookie名

JWT_SECRET仕様

項目	内容
最低長	32バイト
ソースコード記載	禁止
Git管理	禁止
ログ出力	禁止
本番管理	Secret管理機能から注入
未設定時	起動失敗

起動時検証

1. JWT_SECRETが設定されていることを確認
2. JWT_SECRETが32バイト以上であることを確認
3. ACCESS_TOKEN_TTL_MINUTESが0より大きいことを確認
4. Refresh Token有効期限がAccess Token有効期限より長いことを確認
5. BCRYPT_COSTが10以上14以下であることを確認
6. COOKIE_SAME_SITEが有効な値であることを確認

7. 本番環境でCOOKIE_SECUREがtrueであることを確認
8. 条件を満たさない場合はアプリケーションを起動しない

UseCaseファイル分割方針

方針	内容
機能単位	1機能につき1ファイルを基本とする
共通構造体	auth.goなどへUseCase構造体を定義する
共通処理	auth_helpers.goやhelpers.goへ共通処理を定義する
Test	対象ファイル名に対応する_test.goを作成する
Integration Test	複数RepositoryやDBを利用する処理はintegration_test.goを作成する

Model仕様

User

概要

NoteHubを利用するユーザー情報を保持する

構造体

```
type User struct {
    ID          uuid.UUID
    Email       string
    PasswordHash string
    Name        string
    Status      UserStatus
    AuthVersion int
    SuspendedAt *time.Time
    DeletedAt   *time.Time
    CreatedAt   time.Time
    UpdatedAt   time.Time
}
```

フィールド

フィールド	DB型	制約	内容
id	UUID	PK	ユーザーID
email	VARCHAR	UNIQUE、NOT NULL	メールアドレス
password_hash	VARCHAR	NOT NULL	bcryptハッシュ
name	VARCHAR	NOT NULL	表示名
status	VARCHAR	NOT NULL	アカウント状態
suspended_at	TIMESTAMPTZ	NULL可	停止日時
deleted_at	TIMESTAMPTZ	NULL可	論理削除日時
created_at	TIMESTAMPTZ	NOT NULL	作成日時
updated_at	TIMESTAMPTZ	NOT NULL	更新日時
auth_version	INTEGER	NOT NULL	全Access Tokenの失効判定に使用 初期値1

UserStatus

```
type UserStatus stringconst (UserStatusActiveUserStatus="active"UserStatusSuspendedUserStatus="suspended"UserStatusDeletedUserStatus="deleted")
```

状態別制御

status	ログイン	API	WebSocket	Token再発行
active	許可	許可	許可	許可
suspended	拒否	拒否	拒否	拒否
deleted	拒否	拒否	拒否	拒否

Model Method

Method	処理
IsActive	statusがactiveか判定する
IsSuspended	statusがsuspendedか判定する
IsDeleted	statusがdeletedか判定する
Suspend	statusをsuspendedへ変更する
Reactivate	statusをactiveへ戻す
LogicalDelete	statusをdeletedへ変更する

Suspend

```
func (u*User) Suspend(nowtime.Time)error
```

項目	内容
実行可能状態	active
実行不可状態	suspended、deleted
status	suspendedへ変更
suspended_at	現在日時を設定
updated_at	現在日時を設定

Reactivate

```
func (u*User) Reactivate(nowtime.Time)error
```

項目	内容
実行可能状態	suspended
実行不可状態	active、deleted
status	activeへ変更
suspended_at	NULLへ変更
updated_at	現在日時を設定

LogicalDelete

```
func (u*User) LogicalDelete(nowtime.Time)error
```

項目	内容
実行可能状態	suspended
実行不可状態	active、deleted
status	deletedへ変更
deleted_at	現在日時を設定
updated_at	現在日時を設定

項目	内容
email	匿名化した一意な値へ変更
name	削除済みユーザーへ変更
password_hash	ログイン不可能なランダム値へ変更

Email匿名方法

```
deleted_{userId}@deleted.local
```

元のメールアドレスは保持しない

Password仕様

入力条件

項目	条件
必須	必須
最小文字数	8文字
最大文字数	15文字
英字	1文字以上含む
数字	1文字以上含む
空白のみ	不許可
平文保存	禁止
ログ出力	禁止
API Response	出力禁止

bcrypt仕様

項目	内容
アルゴリズム	bcrypt
Cost	12
Salt	bcrypt内部で自動生成
保存先	users.password_hash
保存内容	bcrypt Hash文字列
平文Password	保存しない

Password Hash生成処理

1. Passwordの必須項目を確認
2. Passwordの文字数を確認
3. 英字が含まれていることを確認
4. 数字が含まれていることを確認
5. bcrypt Cost 12でHashを生成
6. 生成したHashのみUserへ設定
7. 平文PasswordをDBやログへ出力しない

Password照合処理

1. EmailからUserを取得
2. 入力されたPasswordを取得
3. bcryptで保存済みpassword_hashと照合
4. 一致した場合はUser.status確認へ進む
5. 不一致の場合は認証失敗を返す

Workspace

構造体

```
typeWorkspacestruct {
    IDuuid.UUID
    Namestring
    HostIDuuid.UUID
    CreatedAttime.Time
    UpdatedAttime.Time
}
```

フィールド

フィールド	DB型	制約	内容
id	UUID	PK	ワークスペースID
name	VARCHAR	NOT NULL	ワークスペース名
host_id	UUID	FK、NOT NULL	ホストユーザーID
created_at	TIMESTAMPTZ	NOT NULL	作成日時
updated_at	TIMESTAMPTZ	NOT NULL	更新日時

ワークスペース利用判定

Workspaceには利用停止用のstatusを持たせない
host_idに紐づくUserのstatusを正として利用可否を判定する

Host status	Workspace
active	利用可能
suspended	利用不可
deleted	利用不可

復帰条件

状態変更	Workspace
suspendedからactive	自動的に利用可能へ戻る
suspendedからdeleted	利用不可を維持する
deletedからactive	状態変更自体を許可しない

WorkspaceMember

構造体

```
typeWorkspaceMemberstruct {
    IDuuid.UUID
    WorkspaceIDuuid.UUID
    UserIDuuid.UUID
    RoleWorkspaceRole
    JoinedAttime.Time
    CreatedAttime.Time
    UpdatedAttime.Time
}
```

フィールド

フィールド	DB型	制約	内容
id	UUID	PK	所属ID
workspace_id	UUID	FK、NOT NULL	ワークスペースID

フィールド	DB型	制約	内容
user_id	UUID	FK、NOT NULL	ユーザーID
role	VARCHAR	NOT NULL	HostまたはMember
joined_at	TIMESTAMPTZ	NOT NULL	参加日時
created_at	TIMESTAMPTZ	NOT NULL	作成日時
updated_at	TIMESTAMPTZ	NOT NULL	更新日時

WorkspaceRole

```
typeWorkspaceRolestringconst (WorkspaceRoleHostWorkspaceRole="host"WorkspaceRoleMemberWorkspaceRole="member"
)
```

制約

制約	内容
重複禁止	workspace_idとuser_idの組み合わせをUNIQUEにする
ホスト数	1ワークスペースにつきHostは1人
ホスト登録	Workspace作成時に自動登録する
ホスト削除	WorkspaceMember削除APIでは削除不可
ホスト移譲	実装対象外

Document

構造体

```
typeDocumentstruct {
    IDuuid.UUID
    WorkspaceIDuuid.UUID
    Titlestring
    Contentstring
    CreatedByuuid.UUID
    UpdatedByuuid.UUID
    CreatedAttime.Time
    UpdatedAttime.Time
}
```

フィールド

フィールド	DB型	制約	内容
id	UUID	PK	ドキュメントID
workspace_id	UUID	FK、NOT NULL	ワークスペースID
title	VARCHAR	NOT NULL	タイトル
content	TEXT	NOT NULL	永続化済み本文
created_by	UUID	FK、NOT NULL	作成ユーザー
updated_by	UUID	FK、NOT NULL	最終更新ユーザー
created_at	TIMESTAMPTZ	NOT NULL	作成日時
updated_at	TIMESTAMPTZ	NOT NULL	更新日時

データ管理方針

データ	保存先	扱い
永続化済み本文	PostgreSQL	保存データ

データ	保存先	扱い
編集中の最新本文	Redis	一時データ
編集履歴	PostgreSQL	DocumentVersionとして保存
接続中ユーザー	メモリまたはRedis	一時データ

DocumentVersion

構造体

```
type DocumentVersion struct {
    ID        uuid.UUID
    DocumentID uuid.UUID
    Content    string
    CreatedBy  uuid.UUID
    VersionType VersionType
    SourceVersionID *uuid.UUID
    CreatedAt    time.Time
}
```

フィールド

フィールド	DB型	制約	内容
id	UUID	PK	バージョンID
document_id	UUID	FK、NOT NULL	ドキュメントID
content	TEXT	NOT NULL	保存時点の本文
created_by	UUID	FK、NOT NULL	保存または復元実行者
version_type	VARCHAR	NOT NULL	履歴種別
source_version_id	UUID	NULL可	復元元バージョンID
created_at	TIMESTAMP TZ	NOT NULL	作成日時

VersionType

値	内容
auto_save	自動保存で作成
manual_save	手動保存で作成
before_restore	復元直前の状態
restore	復元後の状態

RefreshToken

概要

Access Tokenの有効期限が切れた場合に、新しいAccess Tokenを発行するためのToken

Refresh TokenはブラウザのHttpOnly Cookieへ保存し、PostgreSQLには平文ではなくHash化した値のみ保存する
Access Token再発行時はRefresh Tokenをローテーションし、使用済みTokenを再利用できないようにする

構造体

```
type RefreshToken struct {
    ID            uuid.UUID
    UserID        uuid.UUID
    TokenHash     string
    FamilyID      uuid.UUID
    Status        RefreshTokenStatus
    ExpiresAt     time.Time
    CreatedAt     time.Time
    RotatedAt     *time.Time
    RevokedAt     *time.Time
}
```

```

    LastUsedAt    *time.Time
    ReplacedByID *uuid.UUID
    IPAddress     string
    UserAgent     string
}

```

フィールド

フィールド	DB型	制約	内容
id	UUID	PK	Refresh Token ID
user_id	UUID	FK、NOT NULL	Token所有User ID
token_hash	CHAR(64)	UNIQUE、NOT NULL	Refresh TokenのSHA-256 Hash
family_id	UUID	NOT NULL	同一ログイン系統を識別するID
status	VARCHAR(20)	NOT NULL	Refresh Tokenの状態
expires_at	TIMESTAMPZ	NOT NULL	有効期限
created_at	TIMESTAMPZ	NOT NULL	発行日時
rotated_at	TIMESTAMPZ	NULL可	Rotation実行日時
revoked_at	TIMESTAMPZ	NULL可	強制無効化日時
last_used_at	TIMESTAMPZ	NULL可	最終使用日時
replaced_by_id	UUID	NULL可	Rotation後に発行したRefresh Token ID
ip_address	VARCHAR(45)	NULL可	発行元IP Address
user_agent	TEXT	NULL可	発行元User Agent

RefreshTokenStatus

```

typeRefreshTokenStatusstringconst (RefreshTokenStatusActiveRefreshTokenStatus="active"RefreshTokenStatusRotatedRefreshTokenStatus="rotated"RefreshTokenStatusRevokedRefreshTokenStatus="revoked"RefreshTokenStatusExpiredRefreshTokenStatus="expired")

```

状態

status	内容	Token再発行
active	現在利用可能	許可
rotated	Rotationによって使用済み	拒否
revoked	Logoutやアカウント停止で強制無効化済み	拒否
expired	有効期限切れ	拒否

基本仕様

項目	内容
用途	Access Tokenの再発行
生成方法	crypto/rand
乱数長	64バイト
文字列形式	Base64 URL Safe
Padding	なし
開発環境有効期限	30分
本番環境有効期限	14日
ブラウザ保存	HttpOnly Cookie
DB保存	SHA-256でHash化した値のみ
Redis保存	平文を保存しない
ログ出力	平文を出力しない

項目	内容
Rotation	Access Token再発行ごとに実行
再利用検知	実行
Family管理	family_idを使用

保存方針

保存対象	保存先	内容
Refresh Token平文	HttpOnly Cookie	ブラウザからJavaScriptで参照不可
Refresh Token Hash	PostgreSQL	SHA-256 Hashのみ保存
Family ID	PostgreSQL	Tokenの世代関係を管理
Token状態	PostgreSQL	active、rotated、revoked、expired
Access Token	ブラウザメモリ	LocalStorageなどへ保存しない

Refresh Token生成処理

```

1.crypto/randで64バイトのランダム値を生成
2.生成した値をBase64 URL Safe形式へ変換
3.Paddingを付けずに文字列化
4.生成した文字列をRefresh Token平文とする
5.Refresh Token平文をSHA-256でHash化
6.UUID形式のRefresh Token IDを生成
7.UUID形式のfamily_idを生成
8.statusへactiveを設定
9.expires_atへ有効期限を設定
10.Token Hashと管理情報をPostgreSQLへ保存
11.Refresh Token平文をHttpOnly Cookieへ設定
12.平文をDB、Redis、ログへ保存しない

```

Hash生成

```

funcHashRefreshToken(tokenstring)string {hash:=sha256.Sum256([]byte(token))returnhex.EncodeToString(hash[:])
}

```

Hash仕様

項目	内容
アルゴリズム	SHA-256
入力	Refresh Token平文
出力	64文字の16進文字列
DB型	CHAR(64)
用途	Cookieから送られたTokenとの照合

Refresh Tokenは乱数から生成するため、PasswordのようなbcryptではなくSHA-256を使用する

Cookie仕様

Cookie名

```
notehub_refresh_token
```

Cookie属性

属性	開発環境	本番環境
HttpOnly	true	true

属性	開発環境	本番環境
Secure	HTTPS使用時true	true
SameSite	Lax	LaxまたはNone
Path	/api/auth	/api/auth
Max-Age	1800秒	1209600秒
Domain	設定しない	必要な場合のみ設定

SameSite設定

構成	設定
フロントエンドとAPIが同一Site	Lax
フロントエンドとAPIが異なるSite	None
SameSite=Noneの場合	Secure=true必須

初回発行

Login成功時に新しいRefresh Tokenを発行する

初回発行時の設定

フィールド	値
id	新規UUID
user_id	Login User ID
token_hash	SHA-256 Hash
family_id	新規UUID
status	active
expires_at	現在日時から有効期限後
created_at	現在日時
rotated_at	NULL
revoked_at	NULL
last_used_at	NULL
replaced_by_id	NULL
ip_address	Login時のIP Address
user_agent	Login時のUser Agent

Rotation

Access Tokenを再発行するたびに、新しいRefresh Tokenを発行する
使用したRefresh Tokenは直ちにrotatedへ変更する

Rotation処理

- 1.CookieからRefresh Token平文を取得
- 2.Refresh TokenをSHA-256でHash化
- 3.token_hashからRefresh Tokenレコードを取得
- 4.Refresh Tokenを行ロック付きで取得
- 5.statusがactiveであることを確認
- 6.expires_atが有効期限内であることを確認
- 7.Userを取得
- 8.User.statusがactiveであることを確認
- 9.新しいRefresh Token平文を生成
- 10.新しいRefresh TokenをSHA-256でHash化
- 11.新しいRefresh Token IDを生成
- 12.Transactionを開始
- 13.旧Refresh Tokenのstatusをrotatedへ変更
- 14.旧Refresh Tokenのrotated_atへ現在日時を設定
- 15.旧Refresh Tokenのlast_used_atへ現在日時を設定
- 16.旧Refresh Tokenのreplaced_by_idへ新Token IDを設定

```
17.旧Tokenと同じfamily_idで新Refresh Tokenを登録
18.TransactionをCOMMIT
19.新Refresh TokenをHttpOnly Cookieへ設定
20.新Access TokenをResponse Bodyへ設定
21.処理結果を返す
```

Rotation後の旧Token

フィールド	更新値
status	rotated
rotated_at	現在日時
last_used_at	現在日時
replaced_by_id	新Refresh Token ID

Rotation後の新Token

フィールド	値
id	新規UUID
user_id	旧Tokenと同じUser ID
token_hash	新TokenのSHA-256 Hash
family_id	旧Tokenと同じFamily ID
status	active
expires_at	新しい有効期限
created_at	現在日時
rotated_at	NULL
revoked_at	NULL
last_used_at	NULL
replaced_by_id	NULL

再利用検知

一度使用されたRefresh Tokenが再度送信された場合は、盗難または不正利用の可能性があるため再利用として検知する

再利用判定

status	判定
active	正常なRotation処理
rotated	再利用検知
revoked	再利用検知
expired	有効期限切れ

再利用検知処理

```
1.CookieからRefresh Tokenを取得
2.Refresh TokenをSHA-256でHash化
3.Refresh Tokenレコードを取得
4.statusがrotatedまたはrevokedであることを確認
5.Refresh Tokenからuser_idを取得
6.Refresh Tokenからfamily_idを取得
7.Transactionを開始
8.同じfamily_idを持つRefresh Tokenをすべてrevokedへ変更
9.対象Userのauth_versionを1増加
10.AuditLogへ再利用検知を記録
11.TransactionをCOMMIT
12.対象Userの全WebSocket接続を切断
13.Refresh Token Cookieを削除
14.CSRF Token Cookieを削除
15.401 Unauthorizedを返す
```

Family単位の無効化

同じfamily_idを持つTokenは、同一LoginからRotationされた一連のTokenとして扱う

再利用を検知した場合は、現在activeのTokenだけでなく同じFamilyのTokenをすべて無効化する

```
UPDATE refresh_tokens SET
  status='revoked',
  revoked_at= NOW() WHERE family_id= ? AND status IN ('active','rotated')
```

Logout時

Logout時は現在のRefresh Tokenを無効化する

Logout処理

- 1.Refresh Token Cookieを取得
- 2.Refresh TokenをSHA-256でHash化
- 3.Refresh Tokenレコードを取得
- 4.Tokenの所有Userを確認
- 5.Transactionを開始
- 6.Refresh Tokenのstatusをrevokedへ変更
- 7.revoked_atへ現在日時を設定
- 8.AuditLogへLogoutを記録
- 9.TransactionをCOMMIT
- 10.Refresh Token Cookieを削除
- 11.CSRF Token Cookieを削除
- 12.処理結果を返す

重複実行時の扱い

すでにrevokedまたはrotatedのRefresh TokenでLogoutが実行された場合も成功として扱う

同じLogout処理が複数回実行されても、ログアウト済みの状態を維持する

アカウント停止時

対象UserのすべてのRefresh Tokenを無効化する

- 1.対象Userのアカウント停止処理を開始
- 2.対象UserのRefresh Tokenを取得
- 3.statusがactiveまたはrotatedのTokenを対象とする
- 4.対象Tokenのstatusをrevokedへ変更
- 5.revoked_atへ現在日時を設定
- 6.User.auth_versionを1増加
- 7.TransactionをCOMMIT
- 8.以降のToken再発行を拒否

アカウント論理削除時

対象UserのすべてのRefresh Tokenを恒久的に無効化する

処理	内容
Refresh Token	全Tokenをrevokedへ変更
Access Token	auth_version不一致で拒否
Login	User.statusがdeletedのため拒否
Token再発行	User.statusがdeletedのため拒否
Cookie	利用者の次回アクセス時に削除
WebSocket	全接続切断

有効期限切れ

Refresh Tokenのexpires_atが現在日時以前の場合は使用できない

- 1.Refresh Tokenを取得
- 2.expires_atを現在日時と比較
- 3.有効期限内の場合は処理継続
- 4.有効期限切れの場合はstatusをexpiredへ変更
- 5.Refresh Token Cookieを削除
- 6.401 Unauthorizedを返す

Model Method

Method	処理
IsActive	statusがactiveか確認
IsRotated	statusがrotatedか確認
IsRevoked	statusがrevokedか確認
IsExpired	statusまたはexpires_atから有効期限切れを確認
Rotate	Tokenをrotatedへ変更
Revoke	Tokenをrevokedへ変更
MarkExpired	Tokenをexpiredへ変更
CanRefresh	Token再発行に利用できるか確認

CanRefresh

```
func (t*RefreshToken) CanRefresh(nowtime.Time)bool {return t.Status==RefreshTokenStatusActive&&t.ExpiresAt.After(now)&&t.RevokedAt==nil&&t.RotatedAt==nil
}
```

Rotate

```
func (t*RefreshToken) Rotate(replacedByIDuuid.UUID,nowtime.Time,
)error
```

項目	内容
実行可能状態	active
status	rotatedへ変更
rotated_at	現在日時
last_used_at	現在日時
replaced_by_id	新Refresh Token ID

Revoke

```
func (t*RefreshToken) Revoke(nowtime.Time)error
```

項目	内容
対象状態	active、rotated
status	revokedへ変更
revoked_at	現在日時

Repository

```
typeRefreshTokenRepositoryinterface {
    Create(ctxcontext.Context,tx*gorm.DB,refreshToken*model.RefreshToken,
)error

    FindByTokenHash(ctxcontext.Context,tokenHashstring,
) (*model.RefreshToken,error)
```

```

FindByTokenHashForUpdate(ctxcontext.Context, tx*gorm.DB, tokenHashstring,
) (*model.RefreshToken, error)

MarkRotated(ctxcontext.Context, tx*gorm.DB, tokenIDuuid.UUID, replacedByIDuuid.UUID, rotatedAttime.Time,
)error

Revoke(ctxcontext.Context, tx*gorm.DB, tokenIDuuid.UUID, revokedAttime.Time,
)error

RevokeByFamilyID(ctxcontext.Context, tx*gorm.DB, familyIDuuid.UUID, revokedAttime.Time,
)error

RevokeAllByUserID(ctxcontext.Context, tx*gorm.DB, userIDuuid.UUID, revokedAttime.Time,
)error
}

```

Error

ErrorCode	条件	HTTP Status
REFRESH_TOKEN_REQUIRED	CookieにTokenがない	401
REFRESH_TOKEN_INVALID	TokenがDBに存在しない	401
REFRESH_TOKEN_EXPIRED	有効期限切れ	401
REFRESH_TOKEN_REUSED	rotatedまたはrevokedのTokenを再利用	401
ACCOUNT_UNAVAILABLE	Userがactive以外	401
CSRF_TOKEN_INVALID	CSRF Token不一致	403
DATABASE_ERROR	DB処理失敗	500
TOKEN_GENERATION_FAILED	新Token生成失敗	500

完了条件

項目	完了条件
Token生成	crypto/randで64バイトの乱数から生成される
Cookie保存	HttpOnly Cookieへ保存される
DB保存	SHA-256 Hashのみ保存される
平文保護	DB、Redis、ログへ平文が保存されない
Rotation	Access Token再発行時に旧Tokenがrotatedになる
旧Token利用	Rotation後の旧Tokenを再利用できない
再利用検知	同じfamily_idのTokenがすべて失効する
Logout	現在のRefresh Tokenがrevokedになる
アカウント停止	対象Userの全Refresh Tokenが失効する
アカウント削除	Token再発行が恒久的に拒否される
有効期限	期限切れTokenが利用できない

AuditLog

構造体

```

typeAuditLogstruct {
    IDuuid.UUID
    UserID*uuid.UUID
    Actionstring
    TargetUserID*uuid.UUID
    WorkspaceID*uuid.UUID
    DocumentID*uuid.UUID
    IPAddressstring
    Metadata    []byte
    CreatedAttime.Time
}

```

フィールド

フィールド	DB型	内容
id	UUID	監査ログID
user_id	UUID	操作者
action	VARCHAR	操作種別
target_user_id	UUID	操作対象ユーザー
workspace_id	UUID	対象ワークスペース
document_id	UUID	対象ドキュメント
ip_address	VARCHAR	接続元IP
metadata	JSONB	詳細情報
created_at	TIMESTAMPTZ	記録日時

Action

Action	内容
LOGIN	ログイン成功
LOGIN_FAILED	ログイン失敗
LOGOUT	ログアウト
LOGIN_RATE_LIMITED	ログイン制限発動
REFRESH_TOKEN_REUSED	無効化済みToken再利用
WORKSPACE_CREATED	ワークスペース作成
WORKSPACE_UPDATED	ワークスペース更新
WORKSPACE_DELETED	ワークスペース削除
MEMBER_ADDED	メンバー追加
MEMBER_REMOVED	メンバー削除
ACCOUNT_SUSPENDED	アカウント停止
ACCOUNT_REACTIVATED	アカウント停止解除
ACCOUNT_DELETED	アカウント論理削除
DOCUMENT_CREATED	ドキュメント作成
DOCUMENT_DELETED	ドキュメント削除
DOCUMENT_RESTORED	バージョン復元
BACKUP_EXECUTED	バックアップ実行

Repository仕様

Repository一覧

Repository	主な責務
UserRepository	Userの登録、取得、状態更新
WorkspaceRepository	Workspaceの登録、取得、更新、削除
WorkspaceMemberRepository	所属関係の登録、取得、削除
DocumentRepository	Documentの登録、取得、更新、削除
DocumentVersionRepository	編集履歴の登録と取得
RefreshTokenRepository	Refresh Tokenの登録、取得、無効化
AuditLogRepository	監査ログの登録

UserRepository

```
type UserRepository interface {  
    Create(ctx context.Context, tx *gorm.DB, user *model.User) error
```

```

FindByID(ctxcontext.Context, userIDuuid.UUID) (*model.User, error)
FindByIDForUpdate(ctxcontext.Context, tx*gorm.DB, userIDuuid.UUID) (*model.User, error)
FindByEmail(ctxcontext.Context, emailstring) (*model.User, error)
ExistsByEmail(ctxcontext.Context, emailstring) (bool, error)
Update(ctxcontext.Context, tx*gorm.DB, user*model.User) error
}

```

Method	内容
Create	新規Userを登録する
FindByID	User IDから取得する
FindByIDForUpdate	行ロック付きで取得する
FindByEmail	Emailから取得する
ExistsByEmail	Email重複を確認する
Update	User情報を更新する

WorkspaceRepository

```

typeWorkspaceRepositoryinterface {
    Create(ctxcontext.Context, tx*gorm.DB, workspace*model.Workspace) error
    FindByID(ctxcontext.Context, workspaceIDuuid.UUID) (*model.Workspace, error)
    FindByIDForUpdate(ctxcontext.Context, tx*gorm.DB, workspaceIDuuid.UUID) (*model.Workspace, error)
    FindByHostID(ctxcontext.Context, hostIDuuid.UUID) ([]model.Workspace, error)
    FindByUserID(ctxcontext.Context, userIDuuid.UUID) ([]model.Workspace, error)
    Update(ctxcontext.Context, tx*gorm.DB, workspace*model.Workspace) error
    Delete(ctxcontext.Context, tx*gorm.DB, workspaceIDuuid.UUID) error
}

```

Method	内容
Create	Workspaceを登録する
FindByID	Workspace IDから取得する
FindByIDForUpdate	行ロック付きで取得する
FindByHostID	指定ユーザーがホストのWorkspaceを取得する
FindByUserID	指定ユーザーが所属するWorkspaceを取得する
Update	Workspaceを更新する
Delete	Workspaceを削除する

WorkspaceMemberRepository

```

typeWorkspaceMemberRepositoryinterface {
    Create(ctxcontext.Context, tx*gorm.DB, member*model.WorkspaceMember) error
    FindByWorkspaceAndUser(ctxcontext.Context, workspaceIDuuid.UUID, userIDuuid.UUID,
    ) (*model.WorkspaceMember, error)
    FindByWorkspaceID(ctxcontext.Context, workspaceIDuuid.UUID,
    ) ([]model.WorkspaceMember, error)
    Exists(ctxcontext.Context, workspaceIDuuid.UUID, userIDuuid.UUID,
    ) (bool, error)
    Delete(ctxcontext.Context, tx*gorm.DB, workspaceIDuuid.UUID, userIDuuid.UUID,
    ) error
}

```

DocumentRepository

```

typeDocumentRepositoryinterface {
    Create(ctxcontext.Context, tx*gorm.DB, document*model.Document) error
    FindByID(ctxcontext.Context, documentIDuuid.UUID) (*model.Document, error)
    FindByIDForUpdate(ctxcontext.Context, tx*gorm.DB, documentIDuuid.UUID,
    ) (*model.Document, error)
    FindByWorkspaceID(ctxcontext.Context, workspaceIDuuid.UUID,
    ) ([]model.Document, error)
    Update(ctxcontext.Context, tx*gorm.DB, document*model.Document) error
}

```

```

Delete(ctxcontext.Context,tx*gorm.DB,documentIDuuid.UUID)error
}

```

DocumentVersionRepository

```

typeDocumentVersionRepositoryinterface {
    Create(ctxcontext.Context,tx*gorm.DB,version*model.DocumentVersion,
    )error
    FindByID(ctxcontext.Context,versionIDuuid.UUID,
    ) (*model.DocumentVersion,error)
    FindByDocumentID(ctxcontext.Context,documentIDuuid.UUID,
    ) ([]model.DocumentVersion,error)
}

```

RefreshTokenRepository

```

type RefreshTokenRepository interface {
    Create(
        ctx context.Context,
        tx *gorm.DB,
        refreshToken *model.RefreshToken,
    ) error

    FindByTokenHash(
        ctx context.Context,
        tokenHash string,
    ) (*model.RefreshToken, error)

    FindByTokenHashForUpdate(
        ctx context.Context,
        tx *gorm.DB,
        tokenHash string,
    ) (*model.RefreshToken, error)

    MarkRotated(
        ctx context.Context,
        tx *gorm.DB,
        tokenID uuid.UUID,
        replacedByID uuid.UUID,
        rotatedAt time.Time,
    ) error

    Revoke(
        ctx context.Context,
        tx *gorm.DB,
        tokenID uuid.UUID,
        revokedAt time.Time,
    ) error

    RevokeByFamilyID(
        ctx context.Context,
        tx *gorm.DB,
        familyID uuid.UUID,
        revokedAt time.Time,
    ) error

    RevokeAllByUserID(
        ctx context.Context,
        tx *gorm.DB,
        userID uuid.UUID,
        revokedAt time.Time,
    ) error
}

```

AuditLogRepository

```

typeAuditLogRepositoryinterface {
    Create(ctxcontext.Context,tx*gorm.DB,auditLog*model.AuditLog,
    )error
}

```

共通認証と認可

REST API認証処理

1. Access Token取得
2. JWT署名確認
3. 有効期限確認
4. user_id取得
5. User取得
6. User.status確認
7. 認証ContextへUserを保存

Workspaceアクセス処理

1. 認証User取得
2. Workspace取得
3. Workspace Host取得
4. Host.status確認
5. WorkspaceMember取得
6. Role確認
7. UseCase実行

アクセス判定表

User status	Host status	Workspace所属	結果
active	active	あり	許可
active	active	なし	拒否
active	suspended	あり	拒否
active	deleted	あり	拒否
suspended	active	あり	拒否
deleted	active	あり	拒否

ホスト専用処理

以下の処理はWorkspaceMember.roleがhostの場合のみ実行可能とする

処理	ホスト限定
Workspace更新	対象
Workspace削除	対象
メンバー追加	対象
メンバー削除	対象
アカウント停止	対象
アカウント停止解除	対象
アカウント論理削除	対象

Transaction方針

Transactionを使用する処理

UseCase	Transaction内の主な処理
RegisterUser	User登録
Login	Refresh Token登録、AuditLog登録
RefreshToken	旧Token無効化、新Token登録
Logout	Refresh Token無効化、AuditLog登録
CreateWorkspace	Workspace登録、Host登録

UseCase	Transaction内の主な処理
AddMember	WorkspaceMember登録、AuditLog登録
RemoveMember	WorkspaceMember削除、AuditLog登録
SuspendAccount	User停止、Refresh Token全無効化、AuditLog登録
ReactivateAccount	User停止解除、AuditLog登録
LogicalDeleteAccount	User匿名化、Refresh Token全無効化、AuditLog登録
CreateDocument	Document登録、AuditLog登録
DeleteDocument	Document削除、AuditLog登録
RestoreVersion	復元前履歴、Document更新、復元履歴登録

Transaction外で行う処理

処理	理由
WebSocket通知	DB COMMIT後に実行するため
WebSocket切断	DBの状態変更確定後に実行するため
Redisキャッシュ削除	DB COMMIT後に整合性を合わせるため
Cookie設定	HTTP Response処理のため
外部通知	DB Transactionを長時間保持しないため

基本処理順序

1. 事前確認
2. Transaction開始
3. DB更新
4. AuditLog登録
5. COMMIT
6. Redis処理
7. WebSocket処理
8. Response返却

認証機能の実装仕様（AuthUseCase）

AuthUseCase構造体

ファイル

```
uusecase/auth.go
```

構造体

```
type AuthUseCase struct {
    UserRepository      repository.UserRepository
    RefreshTokenRepository repository.RefreshTokenRepository
    AuditLogRepository repository.AuditLogRepository
    PasswordHasher      crypto.PasswordHasher
    RandomTokenGenerator crypto.RandomTokenGenerator
    TokenHasher          crypto.TokenHasher
    JWTManager           crypto.JWTManager
    AccessTokenRevoker   redis.AccessTokenRevoker
    LoginRateLimiter     redis.LoginRateLimiter
    TransactionManager   db.TransactionManager
    WebSocketManager     websocket.Manager
    Clock                Clock
}
```

依存関係

依存先	用途
UserRepository	User登録、取得、状態確認、auth_version更新
RefreshTokenRepository	Refresh Token登録、取得、Rotation、無効化
AuditLogRepository	認証操作の監査ログ保存
PasswordHasher	bcryptによるHash生成と照合
RandomTokenGenerator	Refresh TokenとCSRF Token生成
TokenHasher	Refresh TokenのSHA-256 Hash生成
JWTManager	Access Token発行と検証
AccessTokenRevoker	Access Tokenのjti失効管理
LoginRateLimiter	Login失敗回数とロック状態の管理
TransactionManager	DB Transaction管理
WebSocketManager	強制Logout時のWebSocket切断
Clock	現在日時取得

Access Token生成処理

1. User IDを取得
2. UUID形式のjtiを生成
3. issへJWT_ISSUERを設定
4. audへJWT_AUDIENCEを設定
5. subへUser IDを設定
6. verへUser.auth_versionを設定
7. iatへ現在日時を設定
8. expへ現在日時から15分後を設定
9. JWT_SECRETを使用してHS256で署名
10. Access Token文字列を返す

Access Token Claims

```
typeAccessTokenClaimsstruct {
    AuthVersionint`json:"ver"`jwt.RegisteredClaims
}
```

ユーザー登録（RegisterUserUseCase）

ファイル

```
usecase/auth_register.go
```

目的

新しいUserを登録する

PasswordはbcryptでHash化し、平文を保存しない

Input

```
type RegisterInput struct {
    Email    string
    Name     string
    Password string
    IPAddress string
}
```

Output

```
type RegisterInput struct {
    Email    string
    Name     string
    Password string
    IPAddress string
}
```

入力値検証

項目	条件
Email	必須
Email形式	メールアドレス形式
Email正規化	前後空白削除、英字を小文字化
Name	必須
Name文字数	1文字以上50文字以下
Password	必須
Password文字数	8文字以上15文字以下
Password構成	英字と数字をそれぞれ1文字以上含む
Password空白のみ	不許可

事前条件

条件	エラー
Emailが未登録	処理継続
Emailが登録済み	EMAIL_ALREADY_EXISTS
入力値が不正	VALIDATION_ERROR

処理フロー

- 1.Inputを受け取る
- 2.Emailの前後空白を削除
- 3.Emailを小文字へ変換
- 4.Email形式を検証
- 5.Nameを検証
- 6.Passwordが入力されていることを確認
- 7.Passwordが8文字以上15文字以下であることを確認
- 8.Passwordに英字が含まれていることを確認
- 9.Passwordに数字が含まれていることを確認
- 10.UserRepository.ExistsByEmailでEmail重複を確認
- 11.重複している場合はEMAIL_ALREADY_EXISTSを返す
- 12.bcrypt Cost 12でPassword Hashを生成
- 13.UUID形式のUser IDを生成
- 14.User.statusへactiveを設定
- 15.User.auth_versionへ1を設定
- 16.User.suspended_atへNULLを設定
- 17.User.deleted_atへNULLを設定
- 18.Transactionを開始
- 19.UserRepository.Createを実行
- 20.AuditLogRepository.Createを実行
- 21.TransactionをCOMMIT
- 22.Password平文をResponseへ含めず登録結果を返す

User初期値

フィールド	初期値
id	新規UUID
email	正規化済みEmail
password_hash	bcrypt Cost 12で生成したHash
name	入力された表示名
status	active

フィールド	初期値
auth_version	1
suspended_at	NULL
deleted_at	NULL
created_at	現在日時
updated_at	現在日時

監査ログ

項目	値
action	USER_REGISTERED
user_id	登録されたUser ID
ip_address	InputのIPAddress
created_at	現在日時

Transaction

処理	Transaction内
入力値検証	外
Email重複確認	外
Password Hash生成	外
User登録	内
AuditLog登録	内

Error

ErrorCode	条件	HTTP Status
VALIDATION_ERROR	入力値不正	400
PASSWORD_INVALID	Password要件不一致	400
EMAIL_ALREADY_EXISTS	Email重複	409
PASSWORD_HASH_FAILED	bcrypt Hash生成失敗	500
DATABASE_ERROR	DB処理失敗	500

ログイン (LoginUseCase)

ファイル

```
usecase/auth_login.go
```

目的

EmailとPasswordを照合し、Access Token、Refresh Token、CSRF Tokenを発行する

Input

```
type LoginInput struct {
    Email    string
    Password string
    IPAddress string
    UserAgent string
}
```

Output

```
type LoginInput struct {
    Email    string
    Password string
    IPAddress string
    UserAgent string
}
```

AuthUserOutput

```
typeAuthUserOutputstruct {
    IDuuid.UUID
    Emailstring
    Namestring
    Statusmodel.UserStatus
}
```

認証条件

条件	処理
Userが存在する	Password照合へ進む
Userが存在しない	共通認証エラー
Password不一致	共通認証エラー
statusがsuspended	共通認証エラー
statusがdeleted	共通認証エラー
statusがactive	Token発行へ進む

停止中か削除済みかは利用者へ開示しない

認証失敗時の共通メッセージ

以下の状態を利用者へ区別して返さない

内部状態
Emailが存在しない
Password不一致
Userがsuspended
Userがdeleted

メールアドレスまたはパスワードが正しくありません

処理フロー

1. Inputを受け取る
2. Emailを正規化
3. Email単位のLogin試行制限を確認
4. IP Address単位のLogin試行制限を確認
5. 制限中の場合はLOGIN_RATE_LIMITEDを返す
6. UserRepository.FindByEmailを実行
7. Userが存在しない場合はダミーHashとのbcrypt照合を実行
8. Userが存在する場合は入力Passwordとpassword_hashをbcryptで照合
9. 照合失敗時はEmailとIP Addressの失敗回数を増加
10. User.statusがactiveであることを確認
11. statusがactive以外の場合も共通認証失敗として処理
12. UUID形式のAccess Token jtiを生成
13. User.auth_versionを含むAccess Tokenを生成
14. crypto/randで64バイトのRefresh Tokenを生成
15. Refresh TokenをSHA-256でHash化
16. UUID形式のRefresh Token IDを生成
17. UUID形式のfamily_idを生成
18. Refresh Token.statusへactiveを設定
19. Refresh Token.expires_atを設定

```

20.crypto/randで32バイトのCSRF Tokenを生成
21.Transactionを開始
22.RefreshTokenRepository.Createを実行
23.AuditLogRepository.CreateでLOGINを記録
24.TransactionをCOMMIT
25.EmailとIP AddressのLogin失敗情報を削除
26.Refresh Token平文をHttpOnly Cookieへ設定するためOutputへ格納
27.CSRF TokenをCookieへ設定するためOutputへ格納
28.Access TokenをResponse Bodyへ設定するためOutputへ格納
29.認証結果を返す

```

Access Token

項目	内容
有効期限	15分
保存場所	ブラウザメモリ
Response	JSON Body
LocalStorage	保存禁止
SessionStorage	保存禁止

Refresh Token

項目	内容
開発環境有効期限	30分
本番環境有効期限	24時間
保存場所	HttpOnly Cookie
DB保存	SHA-256 Hashのみ
初回family_id	新規UUID
status	active

Cookie設定

Cookie	属性
notehub_refresh_token	HttpOnly、Secure、SameSite、Path=/api/auth
notehub_csrf_token	JavaScript参照可能、Secure、SameSite

監査ログ

結果	action
ログイン成功	LOGIN
Userなし	LOGIN_FAILED
Password不一致	LOGIN_FAILED
status不正	LOGIN_FAILED
試行制限発動	LOGIN_RATE_LIMITED

Error

ErrorCode	条件	HTTP Status
INVALID_CREDENTIALS	認証情報不正	401
LOGIN_RATE_LIMITED	試行回数超過	429
TOKEN_GENERATION_FAILED	Token生成失敗	500
PASSWORD_COMPARE_FAILED	bcrypt処理失敗	500
DATABASE_ERROR	DB処理失敗	500

アクセストークン再発行(RefreshTokenUseCase)

ファイル

```
usecase/auth_refresh.go
```

目的

有効なRefresh Tokenを利用して新しいAccess TokenとRefresh Tokenを発行する

Input

```
type RefreshInput struct {
    RefreshToken string
    IPAddress    string
    UserAgent    string
}
```

CSRF TokenはCSRFMiddlewareで検証済みとする

Output

```
type RefreshOutput struct {
    AccessToken    string
    ExpiresIn      int
    NewRefreshToken string
    NewCSRFToken   string
}
```

前提条件

条件	処理
Refresh Token Cookieあり	継続
CSRF検証済み	継続
Token HashがDBに存在	継続
statusがactive	正常Rotation
statusがrotated	再利用検知
statusがrevoked	再利用検知
expires_atが有効期限内	継続
User.statusがactive	継続

正常処理

```
1.Refresh Token Cookieを取得
2.Contextのcsrf_validatedがtrueであることを確認
3.Refresh Token平文をSHA-256でHash化
4.Transactionを開始
5.RefreshTokenRepository.FindByTokenHashForUpdateを実行
6.Refresh Tokenが存在することを確認
7.Refresh Token.statusを確認
8.statusがrotatedまたはrevokedの場合は再利用検知処理へ進む
9.expires_atが現在日時より後であることを確認
10.UserRepositoryからUserを取得
11.User.statusがactiveであることを確認
12.UUID形式の新しいAccess Token jtiを生成
13.User.auth_versionを含むAccess Tokenを生成
14.crypto/randで新しいRefresh Tokenを生成
15.新しいRefresh TokenをSHA-256でHash化
16.UUID形式の新しいRefresh Token IDを生成
17.crypto/randで新しいCSRF Tokenを生成
18.旧Refresh Tokenをrotatedへ変更
19.旧Refresh Token.rotated_atへ現在日時を設定
20.旧Refresh Token.last_used_atへ現在日時を設定
21.旧Refresh Token.replaced_by_idへ新Token IDを設定
```

```
22.旧Tokenと同じfamily_idで新Refresh Tokenを登録
23.AuditLogへTOKEN_REFRESHEDを登録
24.TransactionをCOMMIT
25.新Refresh TokenをHttpOnly Cookieへ設定するためOutputへ格納
26.新CSRF TokenをCookieへ設定するためOutputへ格納
27.新Access TokenをResponse Bodyへ設定するためOutputへ格納
28.処理結果を返す
```

再利用検知処理

```
1.Refresh Token.statusがrotatedまたはrevokedであることを確認
2.Refresh Token.user_idを取得
3.Refresh Token.family_idを取得
4.RefreshTokenRepository.RevokeByFamilyIDを実行
5.同じfamily_idのactiveまたはrotated Tokenをrevokedへ変更
6.Userを行ロック付きで取得
7.User.auth_versionを1増加
8.AuditLogへREFRESH_TOKEN_REUSEDを登録
9.TransactionをCOMMIT
10.対象Userの全WebSocketを切断
11.Refresh Token Cookieを削除
12.CSRF Token Cookieを削除
13.REFRESH_TOKEN_REUSEDを返す
```

旧Token更新値

フィールド	値
status	rotated
rotated_at	現在日時
last_used_at	現在日時
replaced_by_id	新Refresh Token ID

新Token初期値

フィールド	値
id	新規UUID
user_id	旧Tokenと同じUser ID
token_hash	新TokenのSHA-256 Hash
family_id	旧Tokenと同じFamily ID
status	active
expires_at	新しい有効期限
created_at	現在日時
rotated_at	NULL
revoked_at	NULL
last_used_at	NULL
replaced_by_id	NULL

Error

ErrorCode	条件	HTTP Status
REFRESH_TOKEN_REQUIRED	Cookieなし	401
REFRESH_TOKEN_INVALID	Tokenなし	401
REFRESH_TOKEN_EXPIRED	有効期限切れ	401
REFRESH_TOKEN_REUSED	使用済みToken再利用	401
ACCOUNT_UNAVAILABLE	Userがactive以外	401
CSRF_TOKEN_INVALID	CSRF検証失敗	403
TOKEN_GENERATION_FAILED	Token生成失敗	500

ErrorCode	条件	HTTP Status
DATABASE_ERROR	DB処理失敗	500

Rotation方針

Refresh処理ごとに新しいRefresh Tokenを発行する
古いRefresh Tokenは再利用できない状態にする

- 1.旧Refresh Token
- 2.revoked_at設定
- 3.新Refresh Token作成
- 4.旧Token.replaced_by_idへ新Token ID設定

処理フロー

- 1.Cookie取得
- 2.CSRF Token照合
- 3.Refresh Token Hash生成
- 4.RefreshTokenRepository.FindByHash
- 5.Token存在確認
- 6.有効期限確認
- 7.revoked_at確認
- 8.User取得
- 9.User.status確認
- 10.新Access Token生成
- 11.新Refresh Token生成
- 12.新Refresh Token Hash生成
- 13.新CSRF Token生成
- 14.Transaction開始
- 15.旧Refresh Token無効化
- 16.新Refresh Token登録
- 17.旧Tokenへreplaced_by_id設定
- 18.COMMIT
- 19.Output返却

Refresh Token再利用検知

無効化済みRefresh Tokenが送信された場合は再利用として扱う

再利用検知時処理

- 1.無効化済みToken検知
- 2.対象User特定
- 3.Transaction開始
- 4.対象Userの全Refresh Token無効化
- 5.AuditLog登録
- 6.COMMIT
- 7.対象Userの全WebSocket切断
- 8.401返却

監査ログ

action	条件
TOKEN_REFRESHED	正常更新
REFRESH_TOKEN_REUSED	無効化済みToken利用
REFRESH_FAILED	不正Token利用

Error

ErrorCode	条件	HTTP Status
CSRF_TOKEN_INVALID	CSRF不一致	403
REFRESH_TOKEN_REQUIRED	Cookieなし	401
REFRESH_TOKEN_INVALID	Token不正	401

ErrorCode	条件	HTTP Status
REFRESH_TOKEN_EXPIRED	有効期限切れ	401
REFRESH_TOKEN_REUSED	無効化済みToken	401
ACCOUNT_UNAVAILABLE	statusがactive以外	401

ログアウト(LogoutUseCase)

ファイル

```
usecase/auth_logout.go
```

目的

現在のAccess TokenとRefresh Tokenを無効化してLogoutする

Input

```
type LogoutInput struct {
    UserID          uuid.UUID
    AccessTokenJTI   uuid.UUID
    AccessTokenExp   time.Time
    RefreshToken     string
    IPAddress        string
}
```

Output

```
type LogoutOutput struct{}
```

Token失効対象

対象	処理
現在のAccess Token	jtiをRedisへ登録
現在のRefresh Token	statusをrevokedへ変更
Refresh Token Cookie	削除
CSRF Token Cookie	削除

処理フロー

```
1.ContextからUser IDを取得
2.ContextからAccess Tokenのjtiを取得
3.ContextからAccess Tokenのexpを取得
4.Contextのcsrf_validatedがtrueであることを確認
5.Refresh Token Cookieを取得
6.Refresh Tokenが存在する場合はSHA-256でHash化
7.Transactionを開始
8.RefreshTokenRepository.FindByTokenHashを実行
9.Refresh Tokenの所有Userが認証Userと一致することを確認
10.Refresh Tokenがactiveまたはrotatedの場合はrevokedへ変更
11.Refresh Token.revoked_atへ現在日時を設定
12.AuditLogへLOGOUTを登録
13.TransactionをCOMMIT
14.Access Tokenの残り有効時間を計算
15.jtiをRedisの失効Keyへ登録
16.Refresh Token Cookieを削除
17.CSRF Token Cookieを削除
18.処理結果を返す
```

Cookie削除

Controllerで以下のCookieを期限切れにする

Cookie	処理
refresh_token	Max-Age=-1
csrf_token	Max-Age=-1

同じ操作を複数回実行しても、結果が変わらないようにする

すでに無効化済みのTokenでLogoutが実行された場合も成功として扱う
ユーザーが安全にログアウト完了できることを優先する

Error

ErrorCode	条件	HTTP Status
CSRF_TOKEN_INVALID	CSRF不一致	403
TOKEN_OWNER_MISMATCH	他ユーザーのToken	403
DATABASE_ERROR	DB処理失敗	500

重複実行時の扱い

すでにRefresh Tokenがrevokedの場合も成功として扱う
Refresh Token Cookieが存在しない場合も、Access Tokenの失効処理を行って成功として扱う
同じLogout処理が複数回実行されてもログアウト済みの状態を維持する

Error

ErrorCode	条件	HTTP Status
CSRF_TOKEN_INVALID	CSRF検証失敗	403
TOKEN_OWNER_MISMATCH	Token所有者不一致	403
AUTH_SERVICE_UNAVAILABLE	Redis障害	503
DATABASE_ERROR	DB処理失敗	500

ログインユーザー情報取得(GetCurrentUserUseCase)

ファイル

```
usecase/auth_me.go
```

目的

現在ログイン中のUser情報を取得する

Input

```
typeGetCurrentUserInputstruct {  
    UserIDuuid.UUID  
}
```

Output

```
typeGetCurrentUserOutputstruct {  
    IDuuid.UUID  
    Emailstring  
    Namestring  
}
```

```
Statusmodel.UserStatus
}
```

処理フロー

```
1.UserID取得
2.UserRepository.FindByID
3.status確認
4.Response生成
```

status制御

status	処理
active	User情報を返す
suspended	認証エラー
deleted	認証エラー

ワークスペース機能の実装仕様(WorkspaceUseCase仕様)

WorkspaceUseCase構造体

ファイル

```
usecase/workspace.go
```

```
typeWorkspaceUseCasestruct {
    UserRepositoryrepository.UserRepository
    WorkspaceRepositoryrepository.WorkspaceRepository
    WorkspaceMemberRepositoryrepository.WorkspaceMemberRepository
    DocumentRepositoryrepository.DocumentRepository
    AuditLogRepositoryrepository.AuditLogRepository
    TransactionManagerdb.TransactionManager
    WebSocketManagerwebsocket.Manager
    RedisDocumentStorerredis.DocumentStore
    ClockClock
}
```

ワークスペース作成(CreateWorkspaceUseCase)

ファイル

```
usecase/workspace_create.go
```

目的

新しいWorkspaceを作成し、作成者をHostとして登録する

Input

```
typeCreateWorkspaceInputstruct {
    UserIDuuid.UUID
    Namestring
    IPAddressstring
}
```

Output

```
typeCreateWorkspaceOutputstruct {
    IDuuid.UUID
    Namestring
    HostIDuuid.UUID
    CreatedAttime.Time
}
```

入力値検証

項目	条件
Name	必須
文字数	1文字以上100文字以下
前後空白	削除する
空白のみ	不許可

事前条件

条件	処理
User.statusがactive	継続
User.statusがsuspended	拒否
User.statusがdeleted	拒否

処理フロー

```
1. User取得
2. User.status確認
3. Workspace生成
4. WorkspaceMember生成 role=host
5. Transaction開始
6. WorkspaceRepository.Create
7. WorkspaceMemberRepository.Create
8. AuditLogRepository.Create
9. COMMIT
10. Output返却
```

Transaction

処理	Transaction内
Workspace登録	内
Host所属登録	内
AuditLog登録	内

Workspace登録のみ成功してHost登録に失敗した場合はRollbackする

Error

ErrorCode	条件	HTTP Status
VALIDATION_ERROR	Name不正	400
ACCOUNT_UNAVAILABLE	User状態不正	403
DATABASE_ERROR	DB処理失敗	500

所属ワークスペース一覧取得(ListWorkspacesUseCase)

ファイル

```
usecase/workspace_list.go
```

目的

ログインUserが所属するWorkspace一覧を取得する

Input

```
typeListWorkspacesInputstruct {  
    UserIDuuid.UUID  
}
```

Output

```
typeWorkspaceListItemstruct {  
    IDuuid.UUID  
    Namestring  
    HostIDuuid.UUID  
    HostNamestring  
    Rolemodel.WorkspaceRole  
    IsAvailablebool  
    UnavailableReasonstring  
    UpdatedAttime.Time  
}
```

一覧表示方針

所属しているWorkspaceは、Hostが停止または削除されていても一覧に表示する
利用不可であることを画面上で確認できるようにする

利用可否

Host status	is_available	unavailable_reason
active	true	空文字
suspended	false	HOST_SUSPENDED
deleted	false	HOST_DELETED

処理フロー

```
1. User状態確認  
2. WorkspaceRepository.FindByUserID  
3. 各WorkspaceのHost取得  
4. WorkspaceMemberからRole取得  
5. 利用可否設定  
6. 一覧返却
```

ワークスペース詳細取得(GetWorkspaceUseCase)

ファイル

```
usecase/workspace_get.go
```

目的

Workspace詳細を取得する

Input

```
typeGetWorkspaceInputstruct {  
    UserIDuuid.UUID  
    WorkspaceIDuuid.UUID  
}
```

Output

```
typeGetWorkspaceOutputstruct {  
    IDuuid.UUID  
    Namestring  
    HostWorkspaceHostOutput  
    Rolemodel.WorkspaceRole  
    CreatedAttime.Time  
    UpdatedAttime.Time  
}
```

アクセス条件

条件	結果
UserがWorkspace所属	次へ進む
Userが未所属	403
Host.statusがactive	取得許可
Host.statusがsuspended	423
Host.statusがdeleted	423

HTTP Status

利用不可Workspaceへのアクセスは423 Lockedを使用する

ワークスペース情報更新(UpdateWorkspaceUseCase)

ファイル

```
usecase/workspace_update.go
```

目的

Workspace名を変更する

Input

```
typeUpdateWorkspaceInputstruct {  
    UserIDuuid.UUID  
    WorkspaceIDuuid.UUID  
    Namestring  
    IPAddressstring  
}
```

Output

```
typeUpdateWorkspaceOutputstruct {  
    IDuuid.UUID  
    Namestring  
    UpdatedAttime.Time  
}
```

実行条件

条件	必須
実行Userがactive	必須
Workspaceが存在	必須
Workspace Hostがactive	必須
実行UserがHost	必須

条件	必須
Nameが有効	必須

処理フロー

```

1.Workspace取得
2.Host状態確認
3.WorkspaceMember取得
4.role=host確認
5.Name検証
6.Transaction開始
7.Workspace更新
8.AuditLog登録
9.COMMIT
10.Output返却

```

Error

ErrorCode	条件	HTTP Status
WORKSPACE_NOT_FOUND	Workspaceなし	404
WORKSPACE_LOCKED	Host状態不正	423
HOST_PERMISSION_REQUIRED	Host以外	403
VALIDATION_ERROR	Name不正	400

ワークスペース論理削除(DeleteWorkspaceUseCase)

ファイル

```
usecase/workspace_delete.go
```

目的

不要になったWorkspaceを論理削除する
Workspaceと関連データは物理削除せず、履歴や参照関係を保持する

Input

```

typeDeleteWorkspaceInputstruct {
    UserIDuuid.UUID
    WorkspaceIDuuid.UUID
    Reasonstring
    IPAddressstring
}

```

Output

```

typeDeleteWorkspaceOutputstruct {
    WorkspaceIDuuid.UUID
    DeletedAttime.Time
}

```

実行条件

条件	内容
実行User	WorkspaceのHost
実行User状態	active
Workspace	存在する
Workspace状態	未削除

条件	内容
Host状態	active
Reason	必須
Reason文字数	1文字以上500文字以下

Workspace追加フィールド

フィールド	DB型	制約	内容
deleted_at	TIMESTAMP TZ	NULL可	論理削除日時
deleted_by	UUID	FK、NULL可	削除実行User
delete_reason	VARCHAR	NULL可	削除理由

論理削除後の状態

機能	処理
Workspace一覧	通常一覧には表示しない
Workspace取得	拒否
Workspace更新	拒否
Member操作	拒否
Document一覧	拒否
Document閲覧	拒否
Document編集	拒否
Version閲覧	拒否
Version復元	拒否
WebSocket接続	拒否
既存WebSocket	全切断
関連データ	保持
復元機能	本要件では対象外

処理フロー

```

1.Workspace取得
2.deleted_at確認
3.Host取得
4.Host.status確認
5.実行UserがHostか確認
6.Reason検証
7.Transaction開始
8.Workspace行ロック取得
9.deleted_at設定
10.deleted_by設定
11.delete_reason設定
12.updated_at更新
13.AuditLog登録
14.COMMIT
15.Workspace配下のWebSocket全切断
16.Redis内のWorkspace関連一時データ削除
17.Output返却

```

Workspace更新内容

```

deleted_at = now
deleted_by = 実行User ID
delete_reason = 入力された理由
updated_at = now

```

Transaction

処理	Transaction内
Workspace取得	外または内
Workspace行ロック	内
Workspace論理削除	内
AuditLog登録	内
WebSocket切断	外
Redis削除	外

WebSocket切断対象

対象	処理
Workspace内の全Document接続	切断
Workspace内の全User	切断
他Workspaceの接続	継続

WebSocket切断理由

WORKSPACE_DELETED

Redis削除対象

```
workspace:{workspaceId}:*
document:{documentId}:content
document:{documentId}:editors
document:{documentId}:autosave
document:{documentId}:connections
```

Workspaceに所属するDocument IDを取得し、対象Documentの一時データを削除する
PostgreSQLへ未保存の最新編集内容がRedisにある場合は、データ消失を防ぐため論理削除前にPostgreSQLへ保存する

論理削除直前の保存

1. Workspace配下のDocument一覧取得
2. Redisに未保存内容があるか確認
3. 未保存内容をPostgreSQLへ反映
4. DocumentVersionを作成
5. Workspace論理削除

AuditLog

項目	値
action	WORKSPACE_DELETED
user_id	実行User ID
workspace_id	対象Workspace ID
ip_address	接続元IP
metadata.reason	削除理由
metadata.delete_type	logical

Repository Method

```
typeWorkspaceRepositoryinterface {
    LogicalDelete(ctxcontext.Context,tx*gorm.DB,workspaceIDuuid.UUID,deletedByuuid.UUID,reasonstring,deletedAttime.Time,
```

```
}error  
}
```

検索条件

```
WHERE deleted_atISNULL
```

論理削除済みWorkspaceを監査目的で取得する場合は、専用Methodを使用する

```
FindByIDIncludingDeleted(ctxcontext.Context,workspaceIDuuid.UUID,  
) (*model.Workspace,error)
```

Error

ErrorCode	条件	HTTP Status
WORKSPACE_NOT_FOUND	Workspaceなし	404
WORKSPACE_ALREADY_DELETED	論理削除済み	409
WORKSPACE_LOCKED	Host状態不正	423
HOST_PERMISSION_REQUIRED	Host以外	403
VALIDATION_ERROR	Reason不正	400
DATABASE_ERROR	DB処理失敗	500

ワークスペース利用可否確認(CheckWorkspaceAccessUseCase)

ファイル

```
usecase/workspace_access.go
```

目的

Workspaceへのアクセス可否を共通判定する

Input

```
typeCheckWorkspaceAccessInputstruct {  
    UserIDuuid.UUID  
    WorkspaceIDuuid.UUID  
    RequiredRole*model.WorkspaceRole  
}
```

Output

```
typeWorkspaceAccessResultstruct {  
    Workspace*model.Workspace  
    Member*model.WorkspaceMember  
    Host*model.User  
}
```

判定順序

- 1.実行User取得
- 2.実行User.status確認
- 3.Workspace取得
- 4.Host取得

5.Host.status確認
6.WorkspaceMember取得
7.RequiredRole確認
8.AccessResult返却

判定表

判定	ErrorCode
実行Userがsuspended	ACCOUNT_SUSPENDED
実行Userがdeleted	ACCOUNT_DELETED
Workspaceなし	WORKSPACE_NOT_FOUND
Hostがsuspended	WORKSPACE_HOST_SUSPENDED
Hostがdeleted	WORKSPACE_HOST_DELETED
Member登録なし	WORKSPACE_ACCESS_DENIED
Role不足	WORKSPACE_PERMISSION_DENIED

利用UseCase

UseCase	利用
GetWorkspace	使用
UpdateWorkspace	使用
DeleteWorkspace	使用
AddMember	使用
RemoveMember	使用
CreateDocument	使用
GetDocument	使用
WebSocket接続	使用
Version一覧	使用
Version復元	使用

メンバー管理機能の実装仕様(MemberUseCase仕様)

MemberUseCase構造体

ファイル

usecase/member.go

```
typeMemberUseCasestruct {  
    UserRepositoryrepository.UserRepository  
    WorkspaceRepositoryrepository.WorkspaceRepository  
    WorkspaceMemberRepositoryrepository.WorkspaceMemberRepository  
    AuditLogRepositoryrepository.AuditLogRepository  
    TransactionManagerdb.TransactionManager  
    WebSocketManagerwebsocket.Manager  
    WorkspaceAccessCheckerWorkspaceAccessChecker  
    ClockClock  
}
```

追加対象ユーザー検索(SearchUserUseCase)

ファイル

```
usecase/member_search.go
```

目的

Workspaceへ追加するUserをEmailで検索する

Input

```
typeSearchUserInputstruct {  
    UserIDuuid.UUID  
    WorkspaceIDuuid.UUID  
    Emailstring  
}
```

Output

```
typeSearchUserOutputstruct {  
    IDuuid.UUID  
    Emailstring  
    Namestring  
    Statusmodel.UserStatus  
}
```

実行条件

条件	内容
実行User	Hostのみ
Workspace	利用可能
検索方式	Email完全一致
自分自身	検索結果から除外
参加済みUser	検索結果から除外
suspended	検索結果から除外
deleted	検索結果から除外

処理フロー

```
1.Workspace Access Check role=host  
2.Email正規化  
3.UserRepository.FindByEmail  
4.User.status確認  
5.自分自身でないか確認  
6.WorkspaceMemberRepository.Exists  
7.未参加の場合のみ返却
```

検索結果

Userが存在しない場合は、同じUSER_NOT_FOUNDとして扱う

メンバー一覧取得(ListMembersUseCase)

ファイル

```
usecase/member_list.go
```

目的

Workspace所属User一覧を取得する

Input

```
typeListMembersInputstruct {
    UserIDuuid.UUID
    WorkspaceIDuuid.UUID
}
```

Output

```
typeWorkspaceMemberOutputstruct {
    UserIDuuid.UUID
    Namestring
    Emailstring
    Statusmodel.UserStatus
    Rolemodel.WorkspaceRole
    JoinedAttime.Time
}
```

実行権限

HostとMemberの双方が取得可能

表示順

```
Host
↓
Member joined_at昇順
```

論理削除User表示

項目	表示
name	削除済みユーザー
email	非表示
status	deleted
role	保存されているRole

メンバー追加((AddMemberUseCase)

ファイル

```
usecase/member_add.go
```

目的

登録済みUserをWorkspaceへMemberとして追加する

Input

```
typeAddMemberInputstruct {
    UserIDuuid.UUID
    WorkspaceIDuuid.UUID
    TargetUserIDuuid.UUID
}
```

```
IPAddrssstring
}
```

Output

```
typeAddMemberOutputstruct {
    UserIDuuid.UUID
    Namestring
    Emailstring
    Rolemodel.WorkspaceRole
    JoinedAttime.Time
}
```

実行条件

条件	内容
実行User	Host
Workspace	利用可能
Target User	active
Target User	実行User本人ではない
既存Member	未登録
追加Role	member固定

処理フロー

```
1.Workspace Access Check role=host
2.Target User取得
3.Target User.status確認
4.自分自身でないか確認
5.既存Member確認
6.WorkspaceMember生成
7.Transaction開始
8.WorkspaceMemberRepository.Create
9.AuditLogRepository.Create
10.COMMIT
11.Output返却
```

競合対策

workspace_id + user_idへUNIQUE制約を設定する
同時追加による重複時はMEMBER_ALREADY_EXISTSへ変換する

Error

ErrorCode	条件	HTTP Status
TARGET_USER_NOT_FOUND	Userなし	404
TARGET_ACCOUNT_UNAVAILABLE	active以外	409
CANNOT_ADD_SELF	Host自身を追加	409
MEMBER_ALREADY_EXISTS	参加済み	409
HOST_PERMISSION_REQUIRED	Host以外	403

メンバー削除(RemoveMemberUseCase)

ファイル

```
usecase/member_remove.go
```

目的

WorkspaceからMemberを削除する

Input

```
typeRemoveMemberInputstruct {  
    UserIDuuid.UUID  
    WorkspaceIDuuid.UUID  
    TargetUserIDuuid.UUID  
    IPAddrstring  
}
```

Output

```
typeRemoveMemberOutputstruct{}
```

実行条件

条件	内容
実行User	Host
Target User	Workspace所属Member
Target Role	member
Host自身	削除不可

処理フロー

```
1.Workspace Access Check role=host  
2.Target WorkspaceMember取得  
3.Target Role確認  
4.Transaction開始  
5.WorkspaceMemberRepository.Delete  
6.AuditLogRepository.Create  
7.COMMIT  
8.Target Userの対象Workspace内WebSocket全切断  
9.完了
```

WebSocket切断理由

```
MEMBER_REMOVED
```

切断範囲

接続	切断
対象Workspace内のDocument接続	切断
他Workspaceの接続	継続
対象User以外の接続	継続

Error

ErrorCode	条件	HTTP Status
MEMBER_NOT_FOUND	所属なし	404
CANNOT_REMOVE_HOST	Hostを削除しようとした	409
HOST_PERMISSION_REQUIRED	Host以外	403

アカウント管理機能の実装仕様(AccountUseCase仕様)

AccountUseCase構造体

ファイル

```
usecase/account.go
```

```
typeAccountUseCasestruct {
    UserRepositoryrepository.UserRepository
    WorkspaceRepositoryrepository.WorkspaceRepository
    WorkspaceMemberRepositoryrepository.WorkspaceMemberRepository
    RefreshTokenRepositoryrepository.RefreshTokenRepository
    AuditLogRepositoryrepository.AuditLogRepository
    TransactionManagerdb.TransactionManager
    WebSocketManagerwebsocket.Manager
    ClockClock
}
```

アカウント停止 (SuspendAccountUseCase)

ファイル

```
usecase/account_suspend.go
```

目的

悪意の疑いがあるMemberのアカウントを一時停止する

Input

```
typeSuspendAccountInputstruct {
    UserIDuuid.UUID
    WorkspaceIDuuid.UUID
    TargetUserIDuuid.UUID
    Reasonstring
    IPAddressstring
}
```

Output

```
typeSuspendAccountOutputstruct {
    TargetUserIDuuid.UUID
    Statusmodel.UserStatus
    SuspendedAttime.Time
    LockedWorkspaceIDs []uuid.UUID
}
```

実行条件

条件	内容
実行User	対象WorkspaceのHost
Target User	対象WorkspaceのMember
Target Role	member
Target status	active
Reason	必須
Reason文字数	1文字以上500文字以下

禁止条件

条件	ErrorCode
Target UserがHost	CANNOT_SUSPEND_HOST
Target Userが実行User本人	CANNOT_SUSPEND_SELF
Target Userがsuspended	ACCOUNT_ALREADY_SUSPENDED
Target Userがdeleted	ACCOUNT_ALREADY_DELETED
対象Workspaceに未所属	TARGET_MEMBER_NOT_FOUND

処理フロー

```
1.Workspace Access CheckをHost権限で実行
2.Target WorkspaceMemberを取得
3.Target UserのRoleがmemberであることを確認
4.Target Userが実行User本人でないことを確認
5.Reasonを検証
6.Target UserがHostのWorkspace一覧を取得
7.Transactionを開始
8.Target Userを行ロック付きで取得
9.Target User.statusがactiveであることを確認
10.Target User.statusをsuspendedへ変更
11.Target User.suspended_atへ現在日時を設定
12.Target User.auth_versionを1増加
13.RefreshTokenRepository.RevokeAllByUserIDを実行
14.Target UserのactiveまたはrotatedのRefresh Tokenをrevokedへ変更
15.AuditLogへACCOUNT_SUSPENDEDを登録
16.TransactionをCOMMIT
17.Target Userの全WebSocketを切断
18.Target UserがHostの全WorkspaceのWebSocketを切断
19.停止結果を返す
```

認証情報の全失効

対象	処理
発行済みAccess Token	auth_version不一致で拒否
Refresh Token	User単位ですべてrevoked
新規Login	status=suspendedで拒否
Token再発行	status=suspendedで拒否
WebSocket	全接続切断

Transaction内処理

処理	Transaction内
User状態更新	内
Refresh Token全無効化	内
AuditLog登録	内
Workspace取得	原則外
WebSocket切断	外

User状態更新

```
status = suspended
suspended_at = now
updated_at = now
```

Refresh Token無効化

対象Userのrevoked_at IS NULLとなっている全Tokenへ現在日時を設定する

対象User本人の切断

全Workspace、全DocumentのWebSocket接続を切断する

切断理由

```
ACCOUNT_SUSPENDED
```

連鎖ロック対象

Target UserがHostを務める全Workspace

ワークスペース自体のDB状態は変更しない

Hostのstatus=suspendedを利用不可判定の正とする

連鎖ロック時の切断

Target UserがHostのWorkspaceに接続中の全Userを切断する

切断理由

```
WORKSPACE_HOST_SUSPENDED
```

AuditLog Metadata

```
{
  "reason": "停止理由",
  "affected_workspace_ids": [],
  "previous_status": "active",
  "new_status": "suspended"
}
```

Error

ErrorCode	条件	HTTP Status
TARGET_MEMBER_NOT_FOUND	対象がMemberでない	404
CANNOT_SUSPEND_HOST	対象がHost	409
ACCOUNT_ALREADY_SUSPENDED	既に停止中	409
ACCOUNT_ALREADY_DELETED	削除済み	409
VALIDATION_ERROR	Reason不正	400
DATABASE_ERROR	DB処理失敗	500

アカウント停止解除（ReactivateAccountUseCase）

ファイル

```
usecase/account_reactivate.go
```

目的

停止中のUserを通常利用可能な状態へ戻す

Input

```
typeReactivateAccountInputstruct {
  UserIDuuid.UUID
  WorkspaceIDuuid.UUID
  TargetUserIDuuid.UUID
  Reasonstring
  IPAddrstring
}
```

Output

```
typeReactivateAccountOutputstruct {
    TargetUserIDuuid.UUID
    Statusmodel.UserStatus
    ReactivatedAttime.Time
    UnlockedWorkspaceIDs []uuid.UUID
}
```

実行条件

条件	内容
実行User	対象Workspace Host
Target User	対象Workspace Member
Target status	suspended
Reason	必須

処理フロー

```
1.Workspaceと実行Host確認
2.Target Member確認
3.Target User取得
4.status=suspended確認
5.Target UserがHostのWorkspace一覧取得
6.Transaction開始
7.Target User行ロック取得
8.statusをactiveへ変更
9.suspended_atをNULLへ変更
10.AuditLog登録
11.COMMIT
12.Output返却
```

復帰方針

Workspaceへ個別ロック状態を保存していないため、Target Userのstatusがactiveになった時点で、Target UserがHostを務めるWorkspaceも自動的に利用可能となる

Token

停止前のRefresh Tokenは再有効化しない

復帰後はUserが再度Loginして新しいTokenを取得する

Error

ErrorCode	条件	HTTP Status
ACCOUNT_NOT_SUSPENDED	suspended以外	409
ACCOUNT_ALREADY_DELETED	deleted	409
TARGET_MEMBER_NOT_FOUND	対象Memberなし	404

LogicalDeleteAccountUseCase

ファイル

```
usecase/account_delete.go
```

目的

悪意が確認された停止中Userを論理削除する

Input

```
typeLogicalDeleteAccountInputstruct {
    UserIDuuid.UUID
    WorkspaceIDuuid.UUID
    TargetUserIDuuid.UUID
    Reasonstring
    IPAddressstring
}
```

Output

```
typeLogicalDeleteAccountOutputstruct {
    TargetUserIDuuid.UUID
    Statusmodel.UserStatus
    DeletedAttime.Time
    LockedWorkspaceIDs []uuid.UUID
}
```

実行条件

条件	内容
実行User	対象Workspace Host
Target User	対象Workspace Member
Target status	suspended
Reason	必須
Host移譲	実施しない

禁止条件

条件	ErrorCode
Target statusがactive	ACCOUNT_MUST_BE_SUSPENDED
Target statusがdeleted	ACCOUNT_ALREADY_DELETED
Target RoleがHost	CANNOT_DELETE_HOST_FROM_OWN_WORKSPACE
Targetが未所属	TARGET_MEMBER_NOT_FOUND

匿名化内容

フィールド	更新値
email	deleted_{userId}@deleted.local
name	削除済みユーザー
password_hash	推測不可能なランダム値のHash
status	deleted
suspended_at	既存値を保持
deleted_at	現在日時
updated_at	現在日時

処理フロー

- 1.Workspace Access CheckをHost権限で実行
- 2.Target WorkspaceMemberを取得
- 3.Target UserのRoleがmemberであることを確認
- 4.Target Userを取得
- 5.Target User.statusがsuspendedであることを確認
- 6.Reasonを検証
- 7.Target UserがHostのWorkspace一覧を取得
- 8.匿名化後のEmailを生成
- 9.ログイン不可能なPassword Hashを生成
- 10.Transactionを開始
- 11.Target Userを行ロック付きで取得
- 12.Target User.emailを匿名化

```
13.Target User.nameを削除済みユーザーへ変更
14.Target User.password_hashをログイン不可能なHashへ変更
15.Target User.statusをdeletedへ変更
16.Target User.deleted_atへ現在日時を設定
17.Target User.auth_versionを1増加
18.RefreshTokenRepository.RevokeAllByUserIDを実行
19.Target Userの全Refresh Tokenをrevokedへ変更
20.AuditLogへACCOUNT_DELETEDを登録
21.TransactionをCOMMIT
22.Target Userの全WebSocketを切断
23.Target UserがHostの全WorkspaceのWebSocketを切断
24.論理削除結果を返す
```

認証情報の永久失効

対象	処理
Access Token	auth_version不一致で拒否
Refresh Token	全Tokenをrevoked
Login	status=deletedで拒否
Token再発行	status=deletedで拒否
WebSocket	全接続切断
アカウント復帰	不許可

Workspace状態

Target UserがHostのWorkspaceは利用不可のまま維持する
Host移譲は行わない

切断理由

```
WORKSPACE_HOST_DELETED
```

既存データ参照

データ	扱い
Document.created_by	User IDを維持
Document.updated_by	User IDを維持
DocumentVersion.created_by	User IDを維持
WorkspaceMember	原則維持
AuditLog	維持
表示名	削除済みユーザーとして表示

Email再利用

匿名化後は元EmailがDBから除去されるため、同じEmailで新規登録可能とする
ただし不正Userの再登録防止する場合は、別途Email Hashや拒否リストを保持する必要がある
現行要件では拒否リストを実装対象外とする

Workspace連鎖ロック仕様

ファイル

```
usecase/account_workspace_lock.go
```

基本方針

Workspaceに個別の停止フラグを持たせない

Host Userのstatusを正として利用可否を判定する

判定処理

```
funcCheckWorkspaceHostStatus(host*model.User,
)error
```

判定表

Host status	結果	ErrorCode
active	利用可能	なし
suspended	利用不可	WORKSPACE_HOST_SUSPENDED
deleted	利用不可	WORKSPACE_HOST_DELETED

判定タイミング

タイミング	判定
Workspace詳細取得	必須
Member一覧取得	必須
Member追加	必須
Member削除	必須
Document一覧取得	必須
Document作成	必須
Document取得	必須
Document更新	必須
Document削除	必須
Version一覧	必須
Version復元	必須
WebSocket接続	必須
WebSocket Event受信	必須または定期再確認

APIエラー

状態	HTTP Status	ErrorCode
Host suspended	423	WORKSPACE_HOST_SUSPENDED
Host deleted	423	WORKSPACE_HOST_DELETED

Response例

```
{
  "error": {
    "code": "WORKSPACE_HOST_SUSPENDED",
    "message": "このワークスペースは現在利用できません"
  }
}
```

削除済みか停止中かを画面に表示するかどうかはUI要件によって決定する
セキュリティ上の理由から共通メッセージへ統一してもよい

WebSocket接続時

- 1.Token検証
- 2.User.status確認
- 3.Document取得
- 4.Workspace取得
- 5.Host取得
- 6.Host.status確認

```
7.WorkspaceMember確認
8.接続許可
```

接続中にHostが停止された場合

```
SuspendAccountUseCase COMMIT
↓
HostのWorkspace一覧取得済み
↓
WebSocketManager.DisconnectByWorkspaceID
↓
全接続へ切断Event送信
↓
Connection Close
```

切断Event

```
{
  "type": "workspace_locked",
  "data": {
    "workspace_id": "uuid",
    "reason": "WORKSPACE_HOST_SUSPENDED"
  }
}
```

Account Controller仕様

AccountController構造体

ファイル

```
controller/account.go
```

```
typeAccountControllerstruct {
    AccountUseCase*usecase.AccountUseCase
}
```

Controller責務

項目	内容
Parameter取得	workspaceId、userId
Body取得	reason
認証User取得	Echo Context
UseCase呼び出し	Suspend、Reactivate、LogicalDelete
Response生成	JSON
Error変換	UseCase ErrorからHTTP Statusへ変換

SuspendAccount API

Endpoint

```
POST /api/workspaces/:workspaceId/members/:userId/suspend
```

Request Body

```
{
  "reason": "不正な編集を繰り返しているため"
```

```
}
```

Response

```
{
  "data": {
    "user_id": "uuid",
    "status": "suspended",
    "suspended_at": "2026-07-17T10:00:00Z",
    "locked_workspace_ids": ["uuid"]
  }
}
```

Controller処理

```
1. 認証User取得
2. workspaceId Parse
3. userId Parse
4. Request Body Bind
5. Validator実行
6. SuspendAccountUseCase実行
7. 200返却
```

ReactivateAccount API

Endpoint

```
POST /api/workspaces/:workspaceId/members/:userId/reactivate
```

Request Body

```
{
  "reason": "調査の結果、問題がないことを確認"
}
```

Response

```
{
  "data": {
    "user_id": "uuid",
    "status": "active",
    "reactivated_at": "2026-07-17T10:30:00Z",
    "unlocked_workspace_ids": ["uuid"]
  }
}
```

アカウント論理削除 (LogicalDeleteAccountUseCase)

Endpoint

```
POST /api/workspaces/:workspaceId/members/:userId/delete
```

Request Body

```
{
  "reason": "悪意ある利用が確認されたため"
}
```

```
}
```

Response

```
{
  "data": {
    "user_id": "uuid",
    "status": "deleted",
    "deleted_at": "2026-07-17T11:00:00Z",
    "locked_workspace_ids": ["uuid"]
  }
}
```

削除API方式

HTTPのDELETEではRequest Bodyの扱いが環境によって不安定になる場合があるため、操作理由を必須とする本機能ではPOST /deleteを採用する

Router仕様

Route一覧

Method	Path	認証	権限	Rate Limit
POST	/api/auth/register	不要	なし	対象
POST	/api/auth/login	不要	なし	対象
POST	/api/auth/refresh	Cookie	なし	対象
POST	/api/auth/logout	必須	なし	対象
GET	/api/me	必須	なし	通常
GET	/api/workspaces	必須	なし	通常
POST	/api/workspaces	必須	なし	対象
GET	/api/workspaces/:workspaceId	必須	Member	通常
PATCH	/api/workspaces/:workspaceId	必須	Host	対象
DELETE	/api/workspaces/:workspaceId	必須	Host	対象
GET	/api/workspaces/:workspaceId/members	必須	Member	通常
GET	/api/workspaces/:workspaceId/users/search	必須	Host	対象
POST	/api/workspaces/:workspaceId/members	必須	Host	対象
DELETE	/api/workspaces/:workspaceId/members/:userId	必須	Host	対象
POST	/api/workspaces/:workspaceId/members/:userId/suspend	必須	Host	対象
POST	/api/workspaces/:workspaceId/members/:userId/reactivate	必須	Host	対象
POST	/api/workspaces/:workspaceId/members/:userId/delete	必須	Host	対象

共通Error仕様

Error構造体

```
typeAppErrorstruct {
    Codestring
    Messagestring
    HTTPStatusint
    Causeerror
}
```

API Response

```
{
  "error": {
    "code": "WORKSPACE_HOST_SUSPENDED",
    "message": "このワークスペースは現在利用できません",
    "request_id": "uuid"
  }
}
```

Error一覧

ErrorCode	HTTP Status	利用者向けメッセージ
PASSWORD_INVALID	400	パスワードの入力内容を確認してください
ACCESS_TOKEN_REQUIRED	401	認証が必要です
ACCESS_TOKEN_INVALID	401	認証情報が正しくありません
ACCESS_TOKEN_EXPIRED	401	ログインし直してください
ACCESS_TOKEN_REVOKED	401	ログインし直してください
REFRESH_TOKEN_REQUIRED	401	ログインし直してください
REFRESH_TOKEN_INVALID	401	ログインし直してください
REFRESH_TOKEN_EXPIRED	401	ログインし直してください
REFRESH_TOKEN_REUSED	401	セキュリティ上の理由によりログアウトしました
ACCOUNT_UNAVAILABLE	401	メールアドレスまたはパスワードが正しくありません
CSRF_TOKEN_REQUIRED	403	リクエストを確認できません
CSRF_TOKEN_INVALID	403	リクエストを確認できません
TOKEN_OWNER_MISMATCH	403	この操作を実行できません
LOGIN_RATE_LIMITED	429	時間を空けて再度お試しください
PASSWORD_HASH_FAILED	500	サーバーエラーが発生しました
PASSWORD_COMPARE_FAILED	500	サーバーエラーが発生しました
TOKEN_GENERATION_FAILED	500	サーバーエラーが発生しました
AUTH_SERVICE_UNAVAILABLE	503	認証サービスを一時的に利用できません

内部情報保護

API Responseへ以下の情報を含めない

非表示対象	内容
SQL文	実行したSQL文やQueryの詳細
DB接続情報	DBのHost、Port、User、Password、接続URL
サーバー内部パス	サーバー上のファイルパスやディレクトリ構成
Stack Trace	エラー発生時の内部Stack Trace
Access Token	認証に使用するAccess Token
Refresh Token	Token再発行に使用するRefresh Token
Password	ユーザーが入力した平文Password
Password Hash	bcrypt等でHash化したPassword
Redis Keyの詳細	Redisで使用している内部Key名や構造

ドキュメント管理機能の実装仕様（DocumentUseCase仕様）

DocumentUseCase構造体

ファイル

```
usecase/document.go
```

構造体

```
typeDocumentUseCasestruct {  
    DocumentRepositoryrepository.DocumentRepository  
    DocumentVersionRepositoryrepository.DocumentVersionRepository  
    WorkspaceRepositoryrepository.WorkspaceRepository  
    WorkspaceMemberRepositoryrepository.WorkspaceMemberRepository  
    UserRepositoryrepository.UserRepository  
    AuditLogRepositoryrepository.AuditLogRepository  
    TransactionManagerdb.TransactionManager  
    RedisDocumentStoreredis.DocumentStore  
    WebSocketManagerwebsocket.Manager  
    WorkspaceAccessCheckerWorkspaceAccessChecker  
    ClockClock  
}
```

依存関係

依存先	用途
DocumentRepository	Document登録、取得、更新、論理削除
DocumentVersionRepository	DocumentVersion登録、取得
WorkspaceRepository	Workspace取得
WorkspaceMemberRepository	Workspace所属確認
UserRepository	User状態確認
AuditLogRepository	監査ログ保存
TransactionManager	DB Transaction管理
RedisDocumentStore	編集中心文の一時保存
WebSocketManager	Document更新通知、接続切断
WorkspaceAccessChecker	Workspace利用可否確認
Clock	現在日時取得

ドキュメント作成（CreateDocumentUseCase）

ファイル

```
usecase/document_create.go
```

目的

Workspace内に新しいDocumentを作成する

Input

```
typeCreateDocumentInputstruct {  
    UserIDuuid.UUID  
    WorkspaceIDuuid.UUID  
    Titlestring  
    IPAddrssstring  
}
```

Output

```
typeCreateDocumentOutputstruct {  
    IDuuid.UUID  
    WorkspaceIDuuid.UUID  
    Titlestring  
    Contentstring  
    CreatedByuuid.UUID  
    CreatedAttime.Time  
}
```

入力値検証

項目	条件
Title	必須
Title文字数	1文字以上100文字以下
前後空白	削除する
空白のみ	不許可
Content初期値	空文字

実行条件

条件	内容
実行User	active
Workspace	存在する
Workspace	未削除
Workspace Host	active
実行User	Workspace所属MemberまたはHost

処理フロー

```
1.Workspace Access Check
2.Title検証
3.Document生成
4.Transaction開始
5.DocumentRepository.Create
6.AuditLogRepository.Create
7.COMMIT
8.Output返却
```

Document初期値

フィールド	値
id	UUID新規生成
workspace_id	InputのWorkspaceID
title	入力されたTitle
content	空文字
created_by	実行User ID
updated_by	実行User ID
created_at	現在日時
updated_at	現在日時
deleted_at	NULL
deleted_by	NULL

Error

ErrorCode	条件	HTTP Status
VALIDATION_ERROR	Title不正	400
WORKSPACE_NOT_FOUND	Workspaceなし	404
WORKSPACE_ACCESS_DENIED	Workspace未所属	403
WORKSPACE_HOST_SUSPENDED	Host停止中	423
WORKSPACE_HOST_DELETED	Host論理削除済み	423
DATABASE_ERROR	DB処理失敗	500

ドキュメント一覧取得 (ListDocumentsUseCase)

ファイル

```
usecase/document_list.go
```

目的

Workspace内のDocument一覧を取得する

Input

```
typeListDocumentsInputstruct {  
    UserIDuuid.UUID  
    WorkspaceIDuuid.UUID  
}
```

Output

```
typeDocumentListItemstruct {  
    IDuuid.UUID  
    Titlestring  
    UpdatedByuuid.UUID  
    UpdatedAttime.Time  
}
```

実行条件

条件	内容
実行User	active
Workspace	未削除
Workspace Host	active
実行User	Workspace所属

取得対象

```
deleted_at IS NULL
```

論理削除済みDocumentは通常一覧へ表示しない

表示順

```
updated_at DESC
```

最新更新Documentを上位に表示する

処理フロー

```
1.Workspace Access Check  
2.DocumentRepository.FindByWorkspaceID  
3.論理削除済みを除く  
4.updated_at降順  
5.Output返却
```

ドキュメント詳細取得（GetDocumentUseCase）

ファイル

usecase/document_get.go

目的

Documentの内容を取得する

Input

```
typeGetDocumentInputstruct {
    UserIDuuid.UUID
    DocumentIDuuid.UUID
}
```

Output

```
typeGetDocumentOutputstruct {
    IDuuid.UUID
    WorkspaceIDuuid.UUID
    Titlestring
    Contentstring
    UpdatedByuuid.UUID
    UpdatedAttime.Time
}
```

処理フロー

```
1.Document取得
2.deleted_at確認
3.Document.workspace_id取得
4.Workspace Access Check
5.Redis編集内容確認
6.Redisに最新内容あり
  → Redis内容を返却
6.Redisに内容なし
  → PostgreSQL内容を返却
```

本文取得優先順位

優先順位	保存先	条件
1	Redis	編集中の最新内容が存在する
2	PostgreSQL	Redisに編集内容が存在しない

Error

ErrorCode	条件	HTTP Status
DOCUMENT_NOT_FOUND	Documentなし	404
DOCUMENT_DELETED	論理削除済み	404
WORKSPACE_ACCESS_DENIED	Workspace未所属	403
WORKSPACE_HOST_SUSPENDED	Host停止中	423
WORKSPACE_HOST_DELETED	Host削除済み	423

ドキュメント情報更新 (UpdateDocumentUseCase)

ファイル

usecase/document_update.go

目的

Documentのタイトルなど、リアルタイム共同編集以外の情報を更新する
本文のリアルタイム編集はWebSocket経由で処理する

Input

```
typeUpdateDocumentInputstruct {
    UserIDuuid.UUID
    DocumentIDuuid.UUID
    Titlestring
}
```

Output

```
typeUpdateDocumentOutputstruct {
    IDuuid.UUID
    Titlestring
    UpdatedAttime.Time
}
```

実行条件

条件	内容
Document	存在する
Document	未削除
Workspace	利用可能
実行User	Workspace所属
Title	有効

処理フロー

```
1.Document取得
2.deleted_at確認
3.Workspace Access Check
4.Title検証
5.Transaction開始
6.Document更新
7.COMMIT
8.WebSocketでDocument情報更新通知
9.Output返却
```

WebSocket Event

```
{
  "type": "document_updated",
  "data": {
    "document_id": "uuid",
    "title": "更新後タイトル"
  }
}
```

ドキュメント論理削除 (DeleteDocumentUseCase)

ファイル

```
usecase/document_delete.go
```

目的

Documentを論理削除する

DocumentとDocumentVersionは物理削除せず保持する

Input

```
typeDeleteDocumentInputstruct {
    UserIDuuid.UUID
    DocumentIDuuid.UUID
    IPAddrstring
}
```

Output

```
typeDeleteDocumentOutputstruct {
    DocumentIDuuid.UUID
    DeletedAttime.Time
}
```

実行条件

条件	内容
Document	存在する
Document	未削除
Workspace	利用可能
実行User	Workspace所属

処理フロー

```
1.Document取得
2.deleted_at確認
3.Workspace Access Check
4.Redisに未保存内容があるか確認
5.未保存内容がある場合はPostgreSQLへ保存
6.DocumentVersion作成
7.Transaction開始
8.Document論理削除
9.AuditLog登録
10.COMMIT
11.対象DocumentのWebSocket全切断
12.Redis一時データ削除
13.Output返却
```

更新内容

```
deleted_at = now
deleted_by = 実行User ID
updated_at = now
```

WebSocket切断理由

```
DOCUMENT_DELETED
```

論理削除後

機能	処理
一覧表示	表示しない
Document取得	拒否
編集	拒否
WebSocket接続	拒否
Version取得	通常利用者は拒否
DocumentVersion	保持

機能	処理
復元	本要件では対象外

編集履歴管理機能の実装仕様（VersionUseCase仕様）

VersionUseCase構造体

ファイル

```
usecase/version.go
```

```
typeVersionUseCasestruct {
    DocumentRepositoryrepository.DocumentRepository
    DocumentVersionRepositoryrepository.DocumentVersionRepository
    WorkspaceAccessCheckerWorkspaceAccessChecker
    TransactionManagerdb.TransactionManager
    RedisDocumentStoreredis.DocumentStore
    WebSocketManagerwebsocket.Manager
    ClockClock
}
```

編集履歴一覧取得（ListVersionsUseCase）

ファイル

```
usecase/version_list.go
```

目的

Documentの編集履歴一覧を取得する

Input

```
typeListVersionsInputstruct {
    UserIDuuid.UUID
    DocumentIDuuid.UUID
}
```

Output

```
typeDocumentVersionListItemstruct {
    IDuuid.UUID
    VersionTypemodel.VersionType
    CreatedByuuid.UUID
    CreatedAttime.Time
}
```

処理フロー

```
1.Document取得
2.Document未削除確認
3.Workspace Access Check
4.DocumentVersionRepository.FindByDocumentID
5.created_at降順
6.Output返却
```

編集履歴詳細取得（GetVersionUseCase）

ファイル

```
usecase/version_get.go
```

目的

指定したDocumentVersionの内容を取得する

Input

```
typeGetVersionInputstruct {  
    UserIDuuid.UUID  
    DocumentIDuuid.UUID  
    VersionIDuuid.UUID  
}
```

Output

```
typeGetVersionOutputstruct {  
    IDuuid.UUID  
    DocumentIDuuid.UUID  
    Contentstring  
    VersionTypemodel.VersionType  
    CreatedByuuid.UUID  
    CreatedAttime.Time  
}
```

検証

条件	処理
Document存在	必須
Document未削除	必須
Workspace利用可能	必須
Workspace所属	必須
Version存在	必須
Version.document_id一致	必須

編集履歴作成（CreateVersionUseCase）

ファイル

```
usecase/version_create.go
```

目的

Documentの現在内容をDocumentVersionとして保存する
主に自動保存処理から使用する

Input

```
typeCreateVersionInputstruct {  
    DocumentIDuuid.UUID  
    Contentstring  
    CreatedByuuid.UUID  
    VersionTypemodel.VersionType  
}
```

処理フロー

```
1.Document取得
2.Document状態確認
3.DocumentVersion生成
4.DocumentVersionRepository.Create
5.完了
```

VersionType

呼び出し元	VersionType
自動保存	auto_save
手動保存	manual_save
復元直前	before_restore
復元処理	restore

編集履歴から復元（RestoreVersionUseCase）

ファイル

```
usecase/version_restore.go
```

目的

指定したDocumentVersionの内容を現在のDocumentへ復元する
復元前の状態も履歴として保存する

Input

```
typeRestoreVersionInputstruct {
    UserIDuuid.UUID
    DocumentIDuuid.UUID
    VersionIDuuid.UUID
}
```

Output

```
typeRestoreVersionOutputstruct {
    DocumentIDuuid.UUID
    VersionIDuuid.UUID
    RestoredAttime.Time
}
```

処理フロー

```
1.Document取得
2.Document状態確認
3.Workspace Access Check
4.復元対象Version取得
5.Version.document_id確認
6.Redisから現在の最新本文取得
7.RedisになければPostgreSQL本文取得
8.Transaction開始
9.現在本文をbefore_restoreとして保存
10.Document.contentを復元対象本文へ更新
11.restore Version作成
12.COMMIT
13.Redis本文を復元後内容へ更新
14.WebSocketで復元Event配信
15.Output返却
```

復元時のVersion

順序	VersionType	内容
1	before_restore	復元直前の本文
2	restore	復元後の本文

Restore Version

source_version_idへ復元元Version IDを保存する

WebSocket Event

```
{
  "type": "document_restored",
  "data": {
    "document_id": "uuid",
    "content": "復元後本文",
    "source_version_id": "uuid"
  }
}
```

接続中の全編集者は受信した本文でエディタ内容を更新する

WebSocket仕様

接続URL

```
GET /ws/documents/:documentId
```

接続時認証

WebSocket接続確立前にAccess Tokenを検証する
平文WebSocket通信は禁止する

環境	通信
本番環境	WSS必須
開発環境	WS許可
本番環境でWS	拒否

接続判定

```
1. WebSocket接続要求
2. Access Token検証
3. User取得
4. User.status確認
5. Document取得
6. Document.deleted_at確認
7. Workspace取得
8. Workspace.deleted_at確認
9. Workspace Host取得
10. Host.status確認
11. WorkspaceMember確認
12. 同時接続数確認
13. 接続許可
```

接続拒否条件

条件	処理
未認証	拒否
Access Token期限切れ	拒否
User suspended	拒否
User deleted	拒否

条件	処理
Workspace未所属	拒否
Workspace論理削除済み	拒否
Host suspended	拒否
Host deleted	拒否
Document論理削除済み	拒否
同時接続上限超過	拒否または既存接続切断

同時接続数制限

同一Userが同一Documentを複数タブで開く場合、同時接続数に上限を設定する

Redis Key

```
ws:document:{documentId}:user:{userId}:connections
```

管理内容

項目	内容
単位	User Document
保存値	Connection ID一覧
上限	環境変数で設定
初期値	2接続
TTL	Connection生存期間に合わせて更新

上限超過時

NoteHubでは新しい接続を拒否する方式を採用する

```
現在接続数取得
↓
上限未満
  → 接続許可
上限以上
  → 新規接続拒否
```

ErrorCode

```
WEBSOCKET_CONNECTION_LIMIT_EXCEEDED
```

WebSocket Client

ファイル

```
websocket/client.go
```

```
typeClientstruct {
    ConnectionIDuuid.UUID
    UserIDuuid.UUID
    DocumentIDuuid.UUID
    WorkspaceIDuuid.UUID
    Conn*websocket.Conn
    Sendchan []byte
}
```

Connection ID

WebSocket接続ごとにUUIDを発行する
同じUserが複数タブで接続した場合も、それぞれ別Connection IDとして管理する

WebSocket Hub

ファイル

```
websocket/hub.go
```

責務

処理	内容
Register	Client接続登録
Unregister	Client接続解除
Broadcast	Document内の全Clientへ配信
DisconnectUser	Userの全接続を切断
DisconnectUserInWorkspace	Userの指定Workspace内接続を切断
DisconnectDocument	Documentの全接続を切断
DisconnectWorkspace	Workspaceの全接続を切断

WebSocket Event共通形式

```
{
  "type": "event_type",
  "data": {},
  "timestamp": "2026-07-17T10:00:00Z"
}
```

Event一覧

Event	方向	内容
connected	Server → Client	接続完了
document_sync	Server → Client	最新本文同期
document_edit	Client → Server	編集内容送信
document_updated	Server → Client	編集内容配信
editor_joined	Server → Client	編集者参加
editor_left	Server → Client	編集者退出
editors_sync	Server → Client	編集者一覧同期
document_restored	Server → Client	Version復元
member_removed	Server → Client	Workspaceから削除
account_suspended	Server → Client	アカウント停止
workspace_locked	Server → Client	Workspace利用停止
document_deleted	Server → Client	Document論理削除
error	Server → Client	エラー通知

初回同期

WebSocket接続成功後、最新Document内容をClientへ送信する

処理フロー

1. 接続認証成功
2. HubへClient登録
3. Redisに最新本文があるか確認

```
4.Redisにあり
  → Redis本文取得
4.Redisになし
  → PostgreSQL本文取得
↓
5.document_sync送信
↓
6.編集者一覧送信
↓
7.他Clientへeditor_joined送信
```

document_sync

```
{
  "type": "document_sync",
  "data": {
    "document_id": "uuid",
    "content": "最新本文",
    "updated_at": "2026-07-17T10:00:00Z"
  }
}
```

Document編集Event

Client送信

```
{
  "type": "document_edit",
  "data": {
    "content": "編集後の本文"
  }
}
```

Server処理

```
1.Event受信
2.User.status確認
3.Workspace Host.status確認
4.Workspace所属確認
5.Content検証
6.Redisへ最新本文保存
7.更新User ID保存
8.自動保存待ち状態更新
9.送信元以外へdocument_updated配信
```

原則

WebSocket Event受信処理からPostgreSQLへ直接Document本文を更新しない
リアルタイム編集内容はRedisへ保存し、自動保存WorkerがPostgreSQLへ反映する

編集者一覧

管理単位

Document

表示情報

項目	内容
user_id	User ID
name	表示名

同一Userが複数タブで接続していても編集者一覧では1人として表示する

退出判定

同一UserのConnectionが1件以上残っている場合はeditor_leftを送信しない
最後のConnectionが切断された時点でeditor_leftを配信する

強制切断

切断条件

条件	切断対象	Reason
Account停止	対象User全接続	ACCOUNT_SUSPENDED
Account論理削除	対象User全接続	ACCOUNT_DELETED
Member削除	対象Workspace内の対象User	MEMBER_REMOVED
Host停止	対象Workspace全User	WORKSPACE_HOST_SUSPENDED
Host論理削除	対象Workspace全User	WORKSPACE_HOST_DELETED
Workspace論理削除	対象Workspace全User	WORKSPACE_DELETED
Document論理削除	対象Document全User	DOCUMENT_DELETED

切断順序

```
DB状態変更
↓
COMMIT
↓
切断Event送信
↓
WebSocket Close
↓
接続情報削除
```

再接続

ネットワーク切断などでWebSocketが切断された場合、Clientは再接続を試行する

再接続時

通常の新規接続と同じ認証と認可を再実行する
以前接続できていたことを理由に接続を許可しない

再接続間隔

```
1秒
↓
2秒
↓
4秒
↓
8秒
↓
最大30秒
```

累乗で待機時間を増加させる

Redis仕様

Redis利用目的

用途	内容
Document Cache	編集中の最新本文
AutoSave	自動保存対象管理
WebSocket Session	接続情報管理

用途	内容
Editors	編集者一覧管理
Rate Limit	Token Bucket
Lock	多重実行防止
CSRF補助	必要な一時情報管理

PostgreSQLを永続データのとし、Redisのみへ重要データを永続保存しない

Document Cache

Key

```
document:{documentId}:content
```

Value

```
{
  "content": "最新本文",
  "updated_by": "uuid",
  "updated_at": "2026-07-17T10:00:00Z"
}
```

Access Token失効管理

Logout済みAccess Tokenを有効期限まで利用不可にする

Redis Key

```
auth:access_token:revoked:{jti}
```

保存内容

項目	内容
Value	1
TTL	Access Tokenの残り有効時間
登録タイミング	Logout成功後
確認タイミング	AuthMiddleware
削除タイミング	TTL経過後にRedisが自動削除

登録処理

1. ContextからAccess Tokenのjtiを取得
2. ContextからAccess Tokenのexpを取得
3. expから現在日時を引いて残り有効時間を計算
4. 残り有効時間が0より大きいことを確認
5. Redisへ失効Keyを登録
6. 残り有効時間をTTLへ設定

検証処理

1. Access Tokenからjtiを取得
2. 失効KeyをRedisで検索
3. Keyが存在しない場合は認証処理を継続
4. Keyが存在する場合はACCESS_TOKEN_REVOKEDを返す

Redis障害時

方針	内容
基本方針	認証を拒否する
理由	Logout済みTokenを誤って許可しないため
HTTP Status	503
ErrorCode	AUTH_SERVICE_UNAVAILABLE

更新タイミング

WebSocketでdocument_editを受信するたびに更新する

AutoSave管理

Key

```
document:{documentId}:autosave
```

Value

項目	内容
document_id	Document ID
updated_by	最終編集User
updated_at	最終編集日時
dirty	PostgreSQL未反映かどうか

編集を受信した場合はdirty=trueとする
PostgreSQLへの保存成功後にdirty=falseとする

Editors管理

Key

```
document:{documentId}:editors
```

管理内容

Documentへ接続しているUser IDを管理する
同一Userの複数ConnectionについてはConnection情報を別途管理する

Lock

同じDocumentに対する自動保存処理の重複実行を防止する

Key

```
lock:document:{documentId}:autosave
```

処理

```
SET key value NX EX ttl
```

Lock取得

結果	処理
成功	自動保存実行
失敗	他処理が実行中のためSkip

処理完了後にLockを解除する

LockにはTTLを設定し、異常終了時に永久Lockにならないようにする

Rate Limit

Token Bucket方式を使用する

管理単位

対象	Key単位
Login	IP Address
Register	IP Address
API	User ID
WebSocket接続	User ID
WebSocket Event	User ID + Document ID

超過時

REST APIは429 Too Many Requestsを返す

WebSocket Eventの場合はerrorEventを返し、悪質な大量送信が継続する場合は接続を切断する

自動保存仕様

AutoSaveBatch

ファイル

```
batch/auto_save.go
```

目的

Redis上の編集Documentを定期的にPostgreSQLへ保存する

基本フロー

1. 自動保存対象Document取得
2. dirty=true確認
3. AutoSave Lock取得
4. Redisから最新本文取得
5. PostgreSQLのDocument取得
6. Transaction開始
7. Document.content更新
8. updated_by更新
9. updated_at更新
10. DocumentVersion作成
11. COMMIT
12. dirty=false更新
13. Lock解除

保存内容

保存先	内容
documents.content	最新本文
documents.updated_by	最終編集User
documents.updated_at	保存日時

保存先	内容
document_versions.content	保存時点の本文
document_versions.created_by	最終編集User
document_versions.version_type	auto_save

自動保存失敗

DB保存失敗時

```
1. ROLLBACK
2. dirty=trueを維持
3. エラーログ記録
4. Lock解除
5. 次回batch実行時に再試行
```

Redisの本文は削除しない

Redis障害時

Redisから本文を取得できない場合はPostgreSQLを更新しない
エラーログを記録し、次回処理で再試行する

保存競合対策

自動保存中に新しい編集が発生する可能性を考慮する
保存開始時のupdated_atまたはRevisionを取得する
保存完了後、Redis側のRevisionが変わっていない場合のみdirty=falseとする

Revision例

```
document:{documentId}:revision
```

編集Eventを受信するたびに加算する

```
編集1 → revision 101
編集2 → revision 102
編集3 → revision 103
```

自動保存開始時

```
revision = 103
```

保存中に編集が発生した場合

```
revision = 104
```

保存完了時

```
開始時 103
現在 104
↓
新しい編集あり
↓
dirty=trueを維持
```

これにより保存処理中に発生した編集内容を未保存のまま消さない

Batch仕様

Batch一覧

Batch	ファイル	処理
AutoSaveBatch	auto_save.go	Redis上の編集内容をPostgreSQLへ自動保存する
DocumentFlushBatch	document_flush.go	未保存のDocument内容をPostgreSQLへ反映する
BackupBatch	backup.go	PostgreSQLのバックアップを実行する
CleanupBatch	cleanup.go	期限切れや不要な一時データを削除する

AutoSaveBatch

ファイル

```
batch/auto_save.go
```

目的

Redis上の編集内容Documentを定期的にPostgreSQLへ保存する

実行方式

項目	内容
実行方法	定期実行
実行単位	Document単位
対象	dirty=trueのDocument
重複実行防止	Redis Lockを使用する
失敗時	次回実行時に再試行する

DocumentFlushBatch

ファイル

```
batch/document_flush.go
```

目的

Redis上に残っている未保存のDocument内容をPostgreSQLへ保存する

実行タイミング

タイミング	実行
サーバー正常終了時	実行
Workspace論理削除前	実行
Document論理削除前	実行
通常の定期保存	AutoSaveBatchを使用

BackupBatch

ファイル

```
batch/backup.go
```

目的

PostgreSQLに保存された永続データを定期的にバックアップする

CleanupBatch

ファイル

batch/cleanup.go

目的

期限切れのTokenや不要なRedis一時データを定期的に削除する

保存対象	保存先	内容
Refresh Token平文	HttpOnly Cookie	ブラウザからJavaScriptで参照不可
Refresh Token Hash	PostgreSQL	SHA-256 Hashのみ保存
Family ID	PostgreSQL	Tokenの世代関係を管理
Token状態	PostgreSQL	active、rotated、revoked、expired
Access Token	ブラウザメモリ	LocalStorageなどへ保存しない

Test仕様

Password Test

Test	確認内容
正常Hash生成	bcrypt Hashが生成される
平文との差異	Hashが平文Passwordと一致しない
Salt確認	同じPasswordでも異なるHashになる
正常照合	正しいPasswordで成功する
不一致	異なるPasswordで失敗する
Cost確認	Cost 12で生成される
7文字	入力エラー
16文字	入力エラー
英字なし	入力エラー
数字なし	入力エラー
ログ保護	平文Passwordがログへ出力されない

Access Token Test

Test	確認内容
正常Token	検証成功
HS256	正常検証
HS256以外	拒否
署名改ざん	拒否
有効期限切れ	拒否
iss不一致	拒否
aud不一致	拒否
sub不正	拒否
jti不正	拒否
Redis失効済み	拒否
auth_version一致	許可
auth_version不一致	拒否
suspended User	拒否

Test	確認内容
deleted User	拒否

Refresh Token Test

Test	確認内容
Token生成	crypto/randで64バイト生成
文字列形式	Base64 URL Safe
DB保存	平文が保存されない
Hash	SHA-256の64文字Hash
初回Login	新しいfamily_idが作成される
正常Rotation	旧Tokenがrotatedになる
新Token	同じfamily_idで作成される
replaced_by_id	新Token IDが保存される
同時Rotation	行ロックにより一方のみ成功
rotated再利用	再利用検知
revoked再利用	再利用検知
Family失効	同一Familyがすべてrevokedになる
auth_version	再利用検知時に1増加
Logout	現在Tokenがrevokedになる
Account停止	Userの全Tokenがrevokedになる
Account削除	Token再発行不可
有効期限切れ	拒否

CSRF Test

Test	確認内容
CookieとHeader一致	処理成功
Cookieなし	403
Headerなし	403
値不一致	403
Refresh API	CSRF検証必須
Logout API	CSRF検証必須
一定時間比較	通常の文字列比較を使用しない

Login Test

Test	確認内容
正常Login	Access TokenとCookieを発行
Emailなし	共通認証エラー
Password不一致	共通認証エラー
suspended	共通認証エラー
deleted	共通認証エラー
失敗回数上限	429
Login成功	失敗回数を削除
Refresh Token	DBにHashのみ保存
Access Token	Response Bodyへ設定
Refresh Token	HttpOnly Cookieへ設定
CSRF Token	通常Cookieへ設定

Logout Test

Test	確認内容
正常Logout	Refresh Tokenがrevokedになる
Access Token	jwtがRedisへ保存される
TTL	Token残り有効時間が設定される
Cookie	Refresh Token Cookieが削除される
CSRF Cookie	削除される
重複Logout	成功として扱う
Logout後API	Access Tokenが拒否される
Logout後Refresh	Refresh Tokenが拒否される

CreateWorkspaceUseCase